Scaling Fully Secure MPC via Robust Recursive Search and Gap Amplification

Matan Hamilis
Reichman University
matan@hamil.is

Ariel Nof
Bar-Ilan University
ariel.nof@biu.ac.il

March 7, 2026

Abstract

We present a new framework for secure computation of arithmetic circuits with *two-thirds honest majority* that lifts semi-honest protocols to full malicious security. Our framework works with any *linear* secret sharing over any finite ring and maintains the type of security of the underlying semi-honest protocol (i.e., computational or information-theoretic). The framework has the following overhead complexity with respect to size $|C|$ of the computed circuit C: it incurs only logarithmic communication overhead in $|C|$ over the cost of the semi-honest protocol, the number of additional rounds is independent of the circuit's size, and the computational work per party is $O(|C|)$ arithmetic operations.

Even when limiting the scope to static adversaries, previous works could only achieve two of these three measures: Either the communication is logarithmic and the computational overhead per party is $O(|C|)$, but the number of additional rounds grows with the circuit's size, or communication is logarithmic and the number of rounds is $O(n)$, but the computational overhead is $O(n\cdot|C|)$. To the best of our knowledge, we are the first to achieve the desired complexity in all three fronts, making the cost of achieving full security in MPC lower than ever.

Our result is achieved via a new verification technique based on a robust recursive search that finds and removes cheaters from the computation.

We further improve our result by reducing costs associated with the number of parties n. While the initial result incurs cubic communication overhead with respect to n and $O(n)$ additional rounds in the worst case, we show how to reduce it to $\sqrt{n^5}$ communication overhead and $O(\sqrt{n\log\log n})$ additional rounds, without sacrificing the computational overhead, which remains $O(|C|)$. This is achieved via a novel technique we call "*gap amplification*" that accelerates the player elimination process, enabling us to reduce the number of calls to the verification subprotocol. This technique is of independent interest as it is general and can be directly applied to any protocol that relies on player elimination.

1 Introduction

1.1 Background

Secure Multi-Party Computation (MPC) [GMW87, Yao86, BGW88, CCD88] enables a set of n mutually mistrusting parties $P=\{P_1,...,P_n\}$ to jointly compute a function over their private inputs while revealing nothing but the output. This should be carried out in the presence of an adversary who may corrupt a subset of the parties, thereby learning their internal state. MPC is found to be

a remarkably general purpose technology which has been applied to a wide range of fields such as privacy preserving machine learning, secure voting, secure auctions and key management. Consequently, much effort has been put into further optimizing existing MPC protocols to improve their performance, widening their security guarantees, and minimizing their costs.

MPC protocols are typically proven secure under a specific *adversarial model*, characterized by the maximal number of parties the adversary may corrupt, which we denote by t. MPC research distinguishes between three possible ranges for t: The most general *dishonest majority* setting where $t<n$, the *honest majority* setting where $t<n/2$ and the *two-thirds honest majority* setting where $t<n/3$. Adversarial models also differ with respect to the type of corruptions, where a *semi-honest* adversary strictly follows the prescribed protocol while a *malicious* adversary may arbitrarily deviate from the protocol. These models also differ regarding the time in which the adversary decides which parties to corrupt: In the presence of a *static* adversary it can only choose the set of parties to be corrupted a-priori, while an *adaptive* adversary can choose which parties to corrupt as the protocol execution progresses. Lastly, in the presence of malicious adversaries, MPC protocols may provide different *security guarantees*. In the context of this work, we only mention static adversaries and consider *full security* where the correct output of the computation is guaranteed, rather than the weaker *security with abort* where the adversary is allowed to force the computation to prematurely terminate without an output. Unfortunately, it has been shown [Cle86] that the "holy grail" of MPC — full security against a malicious adversary — is possible to realize only if an honest majority exists.

Malicious Security Compilers. A highly successful paradigm in the design of efficient MPC protocols is based on first designing a baseline protocol, secure in the presence of semi-honest adversaries and then "compiling" it into a protocol secure even under malicious corruptions. This approach provides a clean separation between the task of designing secure semi-honest protocols and the task of later strengthening them into maliciously secure ones; so that a maliciously secure protocol may benefit from any improvement of the semi-honest baseline. This approach has led to a natural question: What is the minimal overhead necessary to information-theoretically[1] compile a semi-honest protocol into a maliciously secure one? This question hence motivated an ongoing long line of works [GMW87, IPS08, BFO12, LN17, ADEN21, BGIN19, DOS18, DEN24, BGIN21, BGIN22]. While this gap is measured by three different metrics (communication, computation and round complexity), the former has received the majority of the research focus.

Minimizing Communication. A significant milestone towards minimizing the communication overhead of semi-honest to malicious compilers was achieved by Boneh et al. [BBC+19] who presented information-theoretic sublinear distributed zero-knowledge proofs over secret shared data (DZKP). This tool allows a prover to prove a degree-2 statement that is robustly secret shared across a set of parties with sublinear (and even logarithmic) communication in the size of the statement. In the same work [BBC+19], the authors showed that this tool can be leveraged to maliciously secure MPC in the honest-majority setting with sublinear communication overhead using the following blueprint: First a semi-honest protocol is executed up until right before opening the computation output and then a succinct verification step is carried out using DZKP in which each party proves it behaved honestly. This blueprint was later optimized and even adapted to the dishonest majority

[1] We limit the scope to information-theoretic compilers to allow the compiler to retain the same security assumptions used in the semi-honest protocol (if any). Otherwise, communication complexity and rounds, for example, can be trivially reduced to a minimum by employing strong tools such as Fully-Homomorphic Encryption [Gen09]

setting in a long line of works [BGIN19, GSZ20, BGIN21, BGIN22, LDH+23, DEN24].

The Costs of DZKPs. DZKPs have been proven to be a versatile tool for reducing communication complexity in MPC and making the communication cost of malicious security match the cost of a semi-honest protocol. But what about other efficiency metrics such as number of rounds and computational costs?

There are two approaches for using DZKP to verify correctness in MPC. In the first approach, referred to as "the single prover approach", each party proves it acted honestly and sent correct messages. Since there is a single prover who knows the entire statement to be proven, it is possible to make the protocol non-interactive in the random oracle model. On the other hand, this approach comes with a high computational price tag, since the computational cost grows linearly with the number of parties n. (For the sake of clarity, the computational cost of a protocol is defined to be the total number of ring/field operations required by the protocol.) This is due to the fact that each party needs to prove a statement as large as the circuit being evaluated and act as a verifier for each of the other parties, implying that the total number of field/ring operations carried by each party is $\Omega(|C| \cdot n)$. As concrete evidence for why it may become a bottleneck, consider a setting where a number of $n = 100$ parties wish to compute some shallow boolean circuit C of size $|C| = 10^8$. According to [LDH+23] a multiplication of 10^8 items, even on the 61-bit Mersenne prime field, would take 0.08 seconds on a standard laptop, making the protocol computation requiring roughly $0.08 \cdot 100 = 8$ seconds per party. However, in terms of communication, even in the semi-honest protocol, each party in a boolean circuit needs to communicate roughly 10^8 bits (the communication cost in the verification protocol is even lower!). Using a common 1Gbps connection this can be done in less than 0.1 seconds, making the computation almost two orders of magnitude more time consuming than semi-honest communication. While parallelization is a common practice, in some settings it is unlikely to use 100 cores. For example, consider a personal laptop that is physically limited by size, weight and power consumption constraints; adding additional cores to the personal laptop is impossible in practice. Moreover, even in cloud environments, parallelization comes at a cost: more machines are needed in order to parallelize a program, leading to increased operational costs. To summarize, the single prover approach yields low communication and round complexity, but the computational cost is high and may become a bottleneck as the number of parties grows.

A second approach for using DZKPs is to verify the correctness of values on the circuit's wires, instead of verifying the sent messages. Since now none of the parties know the statement to be proven (as these values are kept secret and shared across the parties), the parties *jointly* prove that the values are correct. Hence, in this approach, known as "the distributed prover approach", all the parties emulate together the role of the prover and the verifier in the DZKP. The advantage of this approach is that the DZKP protocol is called once, and thus the computational cost does not grow with the number of parties as before. On the other hand, the parties need to interact to generate the proof and so the communication cost remains the same as in the single prover approach. Moreover, now that a single prover does not exist, it is impossible to apply known techniques to compress the round complexity (unless one is willing to sacrifice communication and compromise on quasilinear computation costs), and the number of rounds is logarithmic in the size of the statement. In the context of MPC, this is translated to $\log(|C|)$ rounds. Unfortunately, for round complexity even a logarithmic additive overhead can be extremely detrimental for efficiency: for passive security the number of rounds grows only with the circuit's *depth*, so if the circuit to be computed is very large but shallow, this logarithmic term can result in adding a number of rounds bigger than the circuit's

depth, thereby damaging the overall efficiency. To summarize, the distributed prover approach yields logarithmic communication overhead and $O(|C|)$ operations per party, but the number of rounds on top of the semi-honest protocol is $O(\log(|C|))$.

All of the above leads to the following natural question:

> *Can we design an information-theoretic semi-honest to full security "compiler" with logarithmic communication overhead, computational overhead per party that does not grow with the number of parties and round overhead that is constant in the computed circuit size?*

The Player Elimination Framework and Its Costs. The standard method to obtain full security is to use player elimination [HMP00]. According to this framework, the circuit is divided into $O(n)$ segments. When computing a segment, the parties proceed to the next segment only after verifying that the current segment was computed correctly. If cheating took place, the parties should be able to identify a pair of conflicting parties (implying that one of them is corrupted), remove them from the computation, and recompute the current segment with less parties. Since an honest majority is assumed, removing such a conflicted pair maintains the honest majority, and we are guaranteed that after t eliminations, only honest parties remain. Furthermore, the division to $O(n)$ segments avoids the need to repeat the computation of the entire circuit each time, reducing the overhead of rerunning the semi-honest computation to be minimal. On the other hand, when using the aforementioned approach of "first-compute-then-verify", the parties need to run a separate verification protocol for each segment. Even if the verification protocol has a constant number of rounds, this implies $\Omega(n)$ rounds added to the semi-honest computation. Moreover, verification protocols typically require at least one round of reaching an agreement across all parties, implying an additive communication overhead of $O(n^2)$. Taking over all segments, this results in an overall $O(n^3)$ communication cost. For a large number of parties, this term quickly becomes a bottleneck. The player elimination process thus seems to inherently limit the scalability of fully secure protocols, allowing working only with a small number of parties. This therefore leads to the following question:

> *Can we design an information-theoretic semi-honest to full security "compiler" where the number of additional rounds is $o(n)$ and the additive communication cost is $o(n^3)$, without sacrificing the sublinear overhead with respect to the circuit size?*

1.2 Our Results

In this work, we answer both questions in the affirmative. First, we present the first framework that lifts semi-honest MPC protocols to full malicious security, in the two-thirds honest majority setting, with communication overhead that is *logarithmic* in the circuit's size, a number of rounds that is *independent* of the circuit's size or depth, and computational overhead per party that is *linear* in its size. We note that if all parties act honestly, then the communication overhead becomes constant with respect to the circuit's size. Only in the worst case, the communication cost may be logarithmic in the circuit's size. Our compiler is statistically secure, works with any linear secret sharing, over any finite ring, and with any semi-honest protocol that is secure up to additive attacks in the two-thirds honest majority setting. Hence, it maintains the type of security of the underlying semi-honest protocol (i.e., computational or information-theoretic). We note that many existing semi-honest protocols have been shown to be secure up to additive attack [GIP15, LN17, CGH+18], and so this requirement is not restrictive. We summarize our first result in the following theorem:

Theorem 1.1. *Let R be a finite ring, let $t \geq 1$ be a security threshold, and let $n = 3t+1$. Then, assuming a protocol π_{mult} for multiplying linearly shared secrets over R that is secure up to an additive attack, there exists an n-party protocol for computing any arithmetic circuit C over R with the following properties:*

- *The protocol is fully secure against malicious adversaries controlling up to t parties with statistical error $\sigma(|C|)$.*
- *If π_{mult} is unconditionally secure, then so is our protocol.*
- *If all parties act honestly, it evaluates a circuit of $|C|$ multiplication gates with $|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) + O(n^3)$ sent ring elements, $\mathsf{depth}(\mathsf{C}) \cdot \mathsf{Round}(\pi_{\mathsf{mult}})$ rounds, and $|C| \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}) + O(|C|)$ computational overhead per party.*
- *If cheating takes place, it evaluates a circuit of $|C|$ multiplication gates with at most $|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) + O(n^3 \cdot \log(|C|/n^2))$ sent ring elements, $\mathsf{depth}(\mathsf{C}) \cdot \mathsf{Round}(\pi_{\mathsf{mult}}) + O(n)$ rounds, and $|C| \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}) + O(|C|)$ computational overhead per party.*

where $\mathsf{Comm}(\pi_{\mathsf{mult}})$, $\mathsf{Round}(\pi_{\mathsf{mult}})$ and $\mathsf{Comp}(\pi_{\mathsf{mult}})$ are the per party communication cost, number of rounds and the computational cost of π_{mult} respectively.

Our protocol works according to the aforementioned paradigm of first computing the circuit with the semi-honest protocol and then running a light-weight verification protocol to verify the correctness of the computation. To obtain full security, we use the player elimination framework [HMP00], where the circuit is divided into different segments that are evaluated and then verified sequentially. If a segment was verified successfully, the parties proceed to the next segment. If verification of a segment fails, a pair of parties is removed from the computation with the guarantee that at least one of them is corrupt, and the current segment is recomputed. Our first technical contribution lies in a new verification protocol that outputs such a pair in case cheating took place. This protocol relies on a technique we call "robust recursive search", which allows an efficient search to be run on the set of triples produced by the semi-honest protocol and detects cheating.

Trading Communication with the Round Overhead. As an intermediate result, we introduce a simpler version of our verification protocol with a number of additional rounds that is logarithmic in the circuit's size instead of constant. However, this version has the advantage that the worst-case communication overhead is $O(n^2 \cdot \log(|C|/n)) + O(n^3)$ instead of $O(n^3 \cdot \log(|C|/n^2))$.

A New Player Elimination Process for Better Scalability. Our verification protocol has essentially a constant number of rounds and the communication cost has a multiplicative term of n^2. However, as mentioned above, due to the player elimination framework that divides the circuit into $\Theta(n)$ segments, the communication cost is cubic in the number of parties and the total round overhead can become $\Omega(n)$ in the worst case which may be prohibitive for large n. Our second result aims at reducing the dependency on n, making the protocol significantly more scalable. Specifically, we introduce a novel technique we call "gap amplification", compatible with the protocol from Theorem 1.1 which accelerates the player removal, thereby allowing us to divide the circuit into fewer segments and call the verification protocol fewer times. Similarly to previous unconditionally secure committee-election protocols [DKMS12, DKM+17] this approach does assume a non-adaptive adversary, but our technique works in the whole range of two-thirds honest majority

(i.e., $n = 3t+1$). Applying this technique on top of the protocol from Theorem 1.1 we obtain the following theorem (strictly improving on Theorem 1.1):

Theorem 1.2. *Let R be a finite ring, let $t \geq 1$ be a security threshold and let $n = 3t+1$. Then, assuming a protocol π_{mult} for multiplying linearly shared secrets over R that is secure up to an additive attack, there exists an n-party protocol for computing any arithmetic circuit C over R with the following security and efficiency properties:*

- *The protocol is fully secure against malicious adversaries controlling up to t parties with statistical error σ.*
- *The protocol has the same type of security (information-theoretic or computational) as π_{mult}.*
- *If all parties act honestly, it evaluates a circuit of $|C|$ multiplication gates with $|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) + O(n^{5/2})$ sent ring elements, $\mathsf{depth}(\mathsf{C}) \cdot \mathsf{Round}(\pi_{\mathsf{mult}})$ rounds, and $|C| \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}) + O(|C|)$ computational overhead per party.*
- *If cheating takes place, it evaluates a circuit of $|C|$ multiplication gates with at most $|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) + O(\sqrt{n^5} \cdot \log(|C|/\sqrt{n^3}))$ sent ring elements, $\mathsf{depth}(\mathsf{C}) \cdot \mathsf{Round}(\pi_{\mathsf{mult}}) + O(\sqrt{n \log\log n})$ rounds, and $|C| \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}) + O(|C|)$ computational overhead per party.*

where $\mathsf{Comm}(\pi_{\mathsf{mult}})$, $\mathsf{Round}(\pi_{\mathsf{mult}})$ and $\mathsf{Comp}(\pi_{\mathsf{mult}})$ are the per party communication cost, number of rounds and the computational cost of π_{mult} respectively.

As part of our contribution, we provide a probabilistic analysis showing the exact trade-offs induced by the protocol parameters. We also provide an analytical analysis showing the concrete ranges of party counts for which we can apply our technique. The analysis shows that even for a moderately small number of parties ($n > 500$), we can apply the optimal parameter choice, while for fewer parties slightly less optimal choices are still available. Finally, we conjecture and leave for future work further optimizations to make our analysis applicable and practical to fewer parties as well.

1.3 Related Work

We present a detailed comparison with previous works that achieve full security with sublinear communication overhead in Table 1. All work in the table relies on DZKP, except for [DEN22]. However, their technique is tailored for replicated secret sharing, where the share size grows exponentially with the number of parties. Hence, their protocol is efficient for a small number of parties only. As can be seen, among all works that extend to any number of parties, our work offers the best mix of efficiency metrics. On the other hand, we require two-thirds honest majority, while some works in the table support weak honest majority as well.

We next mention a few additional noteworthy works in the two-thirds honest-majority setting. Furukawa and Lindell [FL19] designed a Shamir sharing based protocol that achieves security with abort and a constant-round verification protocol executed at the end. Nevertheless, their verification technique is the starting point of our protocol. There is a large body of work on achieving *perfect* security when $t < n/3$ with linear communication complexity (see [BTH08, GLS19, AAPP23] and references within). The focus in these works is the *overall* asymptotic communication complexity, but the additive overhead beyond the underlying semi-honest protocol is not sublinear as in our work.

	(n,t)	Additive Overhead			Secret sharing scheme
		Comm	**Comp**	**Round**	
Boyle et al. [BGIN19]	(3,1)	$O(\log(\|C\|))$	$O(\|C\|)$	$O(n)$	Replicated
Boyle et al. [BGIN20]	$(2t+1,t)$	$O(n^3 \cdot \log^2(\|C\|/n))$	$O(n \cdot \|C\|)$	$O(n \cdot \log(\|C\|/n))$	Replicated
Goyal et al. [GSZ20]	$(2t+1,t)$	$O(n^3 \cdot \log(\|C\|/n))$	$O(\|C\|)$	$O(n \cdot \log(\|C\|/n))$	Any linear scheme
Dalskov et al. [DEN22]	$(3t+1,t)$	$O(n^4)$	$O(n \cdot \|C\|)$	$O(n)$	Replicated
Dalskov et al. [DEN24]	$(3t+1,t)$	$O(n^3 \cdot \log(\|C\|/n))$	$O(n \cdot \|C\|)$	$O(n)$	Any linear scheme
Dalskov et al. [DEN24]	$(2t+1,t)$	$O(n^3 \cdot \log(\|C\|/n))$	$O(n \cdot \|C\|)$	$O(n)$	Replicated
Section 4	$(3t+1,t)$	$O(n^2 \cdot \log(\|C\|/n) + n^3)$	$O(\|C\|)$	$O(n \cdot \log(\|C\|/n))$	Any linear scheme
Section 5 (Thm. 1.1)	$(3t+1,t)$	$O(n^3 \cdot \log(\|C\|/n^2))$	$O(\|C\|)$	$O(n)$	Any linear scheme
Section 6+5 (Thm. 1.2)	$(3t+1,t)$	$O(\sqrt{n^5} \cdot \log(\|C\|/\sqrt{n^3}))$	$O(\|C\|)$	$O(\sqrt{n \log\log n})$	Any linear scheme

Table 1: Comparison of the overhead to achieve full security when computing a circuit of size $|C|$ by n parties. The comparison includes only works that introduce sublinear communication overhead. We measure the communication, computation and number of rounds overhead *in the worst case* (i.e., when cheating took place). For works that use replicated secret sharing, the communication and computation should be multiplied with $\binom{n}{t}$ due to the share size.

The works of [DKMS12, DKM+17] presented fully secure protocols based on committee election but require $t < n(1/3 - \epsilon)$ for an arbitrarily small constant $\epsilon > 0$. Finally, there exist several works that are designed for four parties and one corruption that achieve full security, such as [DEK21] and [KPRS22].

1.4 Technical Overview

We have two main technical contributions compatible with each other. First, we design a new efficient verification protocol via a technique we call *robust recursive search* which outputs a pair of conflicting parties in case cheating took place. We call this pair "a semi-corrupt pair" since one of the parties is guaranteed to be corrupt. Our second contribution is a technique we call *gap amplification* that accelerates the player removal, which means we can call the verification protocol fewer times, reducing both the communication cost and the number of rounds.

Let us start with an overview of our verification protocol. At a high level, our verification works by running a carefully designed recursive search over a set of multiplication triples, where in each iteration the parties carry out a simple constant-cost check. The first step of our protocol is the simple check from [FL19]. The observation made in [FL19] is that since sharings with threshold $2t$ are robust when $t < n/3$ (i.e., the shares of the honest parties alone suffice to compute the secret), it is possible to take all multiplication triples (x_k, y_k, z_k) generated during the circuit's computation, and let the parties locally compute a *robust* secret sharing of the random linear combination $u = \sum_{k=1}^{m} \alpha_k \cdot (z_k - x_k \cdot y_k)$. where the α_k's are random coefficients sampled by the parties and m is the number of multiplication gates in the circuit (or segment). That is, the parties locally compute a sharing of u with threshold $2t$. Then, it remains to securely check that $u = 0$, which can be done efficiently with constant cost since the secret sharing is robust. Thus, if $u = 0$, we are guaranteed (except for a small probability) that no cheating took place and all values on output wires of multiplication gates are correct.

The above suffices if we are only interested in security with abort. However, our goal is to achieve

full security, which requires us to find a semi-corrupt pair in case the check fails. Unfortunately, concluding that $u \neq 0$ only means that somewhere in the semi-honest computation someone cheated, but does not give us any information on the identity of the cheater. Moreover, in the protocol, the check can fail not only since $u \neq 0$, but also due to corrupted parties sending incorrect shares when checking the value of u (resulting in an inconsistent sharing). This is possible since the robustness of this sharing only means that corrupted parties cannot change the secret, but not that it cannot prevent its reconstruction. Note that in the first case when $u \neq 0$, we know that cheating took place in the semi-honest protocol. In the second case where u cannot be reconstructed, however, it is possible that cheating took place in the verification only. Nevertheless, in both cases, we don't know who cheated and where the cheating took place. The goal is therefore to run a protocol to locate a semi-corrupt pair, which is then removed from the computation.

First Protocol. In the first version of our protocol, the parties run a binary recursive search over the triples, until locating a single multiplication triple for which the parties perform further inquiry with a small constant cost. Specifically, in each step of the search, the parties split the triples into two equal-size subsets and check equality to 0 for the random linear combination associated with each subset. If the check fails for some subset, the parties recursively repeat the process on this subset. Intuitively, the communication cost is dominated by $\log(m)$ secret reconstructions for a segment of size m, implying $n^2 \cdot O(\log m)$ communication. The computational cost is dominated by computing the random linear combination, implying $O(m)$ ring operations. While the main idea is simple and intuitive, some subtleties must be considered.

Preventing a "Slowdown" Attack. Assume that all triples are correct and $u = 0$, but the reconstruction fails due to inconsistency. According to our protocol, the parties proceed to the binary search. Now, in the next step, the parties split the triples into two halves and check each of them separately. Note that since all triples are correct, the two checks may succeed. In this case, one would argue that we can output accept and halt. Such a strategy will indeed yield a correct output. However, it enables the adversary to mount an attack that causes the verification protocol to run in a *linear* time! Specifically, the adversary can cause all reconstructions to fail, until we reach a leaf of the tree and then allow the check to succeed. In this case, the parties will each time return to the head of a path that has not been scanned yet, and restart the search from that point. As a result, all nodes in the tree will be traversed, resulting in a linear search time. To avoid this problem, our protocol works by the following approach. Assume that the check whether $u = 0$ has failed and denote the two partial sums to be checked in the next round by u_L and u_R, such that $u_L + u_R = u$. Instead of checking the equality to 0 of both u_L and u_R, the protocol checks only that $u_L = 0$. If this is not the case, the parties proceed to next iteration with u_L. However, if $u_L = 0$, the parties compute directly the shares of u_R by taking $u - u_L$ and proceed to next iteration with u_R. The observation is that if $u \neq 0$ or if the shares published for u are inconsistent, and at the same time, $u_L = 0$ and all its shares are consistent, then $u - u_L$ will inherit the properties of u. Hence, we will never need to go back to an upper layer in the tree, and the search is guaranteed to end with a triple which is either incorrect or that its check has failed due to inconsistency.

The Necessity of Additive Security and Randomization. As explained above, a crucial property of our binary search is that given the value on a node and one of its descendants, the parties are able to locally compute their shares of the value on the other descendant. This implies

revealing the shares held by the parties on each node being examined. Unfortunately, using this approach directly is not secure. To see this, recall that on each node we reveal $\sum_{k\in T}\alpha_k\cdot(z_k-x_k\cdot y_k)$ for some subset $T\in[m]$ to the parties. This may reveal some information about private honest parties' shares. One could, of course, run a secure protocol to check equality to 0 (as done in [FL19]), but then we will not be able to compute the value on the other child node as we desire.

To overcome this, we show that assuming additive security for the semi-honest protocol and, in addition, randomizing the shares held by the parties, suffices for revealing the values on each node while maintaining both security and the required search property. In more detail, in order for the protocol to be simulatable (and thus secure), the ideal-world simulator must know the opened value. This is obtained by the additive security assumption which implies that the simulator can extract the additive error on the output wire of each multiplication. That is, for each k, it knows $d_k=z_k-x_k\cdot y_k$, and so it can compute the value of the reconstructed linear combination. Then, it needs to choose the honest parties' shares given the reconstructed value and the corrupted parties' shares. To this end, we randomize the opened sharing each time by adding a sharing of 0. Hence, the honest parties' shares are truly random and the simulator can choose random shares that together with the corrupted parties' shares will reconstruct to the desired value. At the same time, the parties can use these shares to locally compute shares of the value on the second child node, as adding zero does not change the value of the secret itself.

Second Protocol: Balanced Recursive Search and DZKP. The protocol described so far achieves the desired complexity in terms of communication and computation. However, the number of rounds is logarithmic in the size of the verified segment. Our second protocol achieves a constant number of rounds, without changing the other complexity measures. In this protocol, we combine the search idea from above with a verification based on sublinear distributed zero-knowledge proofs (DZKP) to obtain the best of both worlds. Our above method has the advantage of the computation overhead being $O(|C|)$, whereas DZKP based verification has the advantage that it can be carried out in a constant number of rounds (in the random oracle model). On the other hand, when verifying the correctness of all multiplications via DZKP, each party proves it behaved honestly to the other parties, implying that the computational overhead is $O(|C|\cdot n)$. The idea is therefore to run a recursive search as in the first protocol, but to halt when locating a subset of triples of size $\frac{|C|}{n^2}$. Then, the parties verify the correctness of this set using DZKP based verification, but since the statement to verify is of size $\frac{|C|}{n^2}$, the computational overhead is $O(\frac{|C|}{n^2}\cdot n^2)=O(|C|)$ as required (the circuit is divided into n segments of size $|C|/n$ and in each segment we run the search until reaching a subset of size $|C|/n^2$).

Note that once the desired size has been reached we need to take into account two cases. Denote by $\{x_k,y_k,z_k\}_{k\in T}$ the set of triples to verify with $|T|=\frac{|C|}{n^2}$ and let α_k be the random coefficient chosen for the kth triple. In the first case, we have $\sum_{k\in T}\alpha_k\cdot(z_k-x_k\cdot y_k)\neq 0$. In this case, we would like to use DZKP to prove that messages sent in the semi-honest protocol to compute these triples are correct. Since we know that one of the triples is incorrect, a proof of one of the parties should fail and the parties will output a semi-corrupt pair. In the second case, the $2t$-sharing of $\sum_{k\in T}\alpha_k\cdot(z_k-x_k\cdot y_k)$ is inconsistent. In this case, we want each party to prove it published the correct share. Since the shares are inconsistent, one of the proofs is going to fail, resulting in identifying a semi-corrupt pair. Hence, we design two DZKP-based subprotocols, one for each of the two cases. In each case, we show how each party's message (in the verification or in the semi-honest protocol) can be viewed as a degree-2 computation over robustly shared inputs, and hence the DZKP

machinery can be applied, with the desired complexity measures. Note that implementing a DZKP protocol to prove that semi-honest protocol messages are correct, depends on the specific semi-honest protocol used. We show a DZKP-based verification protocol for checking the correctness of the popular DN multiplication protocol [DN07]; our realization follows the approach of [DEN24].

Finally, note that the recursive search until locating a set of size $|C|/n^2$ does not have to be carried out with radix 2. If we use a binary search, then the number of rounds is $\log n$. To obtain a constant number of rounds, it is possible to use for example radix $\sqrt{n}$. In this case, the parties reach the desired set size in two rounds, while the communication in these rounds is independent of the circuit's size. Overall, the number of rounds is constant, the communication cost is logarithmic in $|C|/n^2$ and computational work is $O(|C|)$.

Scalability in n. While the protocol presented above achieves the goal of having low overhead with respect to the circuit size, the overhead with respect to the number of parties is still prohibitively large. In particular, the communication cost is *cubic* in the number of parties, and the number of rounds added to the computation in the worst case is $\Omega(n)$. The main reason for this complexity is the player elimination framework. Since we divide the circuit into $O(n)$ segments and run the verification protocol for each segment, then in the worst case where cheating takes place, the communication overhead is $n \cdot O(n^2 \cdot \log(|C|/n^2))$ (overall, across all parties together) and the number of additional rounds is $O(n)$. Alternatively, if we decide to use fewer segments, then it will result in an even worse situation: we will need to recompute a larger portion of the circuit each time.

Towards a New Solution. As explained above, the large overhead is due to the partition of the circuit into $O(n)$ segments. The reason for using $O(n)$ segments is that each time we remove one corrupted party and hence only after $t = O(n)$ removals, we are guaranteed that only honest parties remain. By using $O(n)$ segments we ensure that even in the worst case, each gate will be computed approximately once.

Clearly, if we are able to remove many corrupted parties at once, and not just a single pair each time, we will be able to partition the circuit into fewer segments to begin with and avoid the $O(n)$ overhead. A possible direction to address the scalability issue is to take the approach of committee-selection [DKMS12, BGT13]: by sampling a committee of some small size k, we can remove many parties, including corrupted ones, at once. However, for this to work, we need that with overwhelming probability, the committee contains at most $k/3$ corrupted parties. Otherwise, the security of the protocol is completely broken. Unfortunately, this holds only when the fraction of corrupted parties is $1/3-\epsilon$ for some constant positive "gap" $\epsilon > 0$. Slightly more formally, we say that a set of n parties with at most t of them are corrupted has a two-thirds security gap $g(n)$ if $t < n/3 - g(n)$. Therefore, we need the gap $g = \Omega(n)$ for the committee selection technique to work. Unfortunately, in our case we do not have such a gap and there is no constant $\epsilon > 0$ such that $t < n(1/3-\epsilon)$, as we have $n = 3t+1$.

Gap Amplification. To overcome this limitation, we introduce a new technique we call *gap amplification* that allows us to amplify the gap between the number of corrupted parties and the threshold of $n/3$. Recall that as part of our protocol we eliminate *pairs* of parties, where one of them is guaranteed to be a cheater. We make the following key observation: *by eliminating pairs we in fact amplify the gap!* For example, for $t = n/3-1$ after $n/6$ eliminations of pairs of parties, we are left with $n' = 2n/3$ parties, where at most $t' = n/3 - n/6 - 1 = n/6 - 1$ are corrupted. Hence, after $n/6$ eliminations of pairs of parties only, we have that the fraction of corrupted parties is

$t'/n' < \frac{n/6-1}{2n/3} < 1/4$— thereby amplifying the gap from 0 to $n/3 - n/4 = n/12$ which suffices for applying the committee selection technique. Unfortunately, while this naive approach does amplify the gap, it does so with $\Omega(n)$ round overhead, missing the original goal of reducing the worst-case round complexity. To address this, we notice that existing probabilistic analysis of existing committee based approaches aim for a tiny committee of size $k(n) = O(\log n)$ and therefore require a gap $g = \Omega(n)$. This opens the door for a possibly interesting and natural trade-off:

> *For what other combinations of gaps $g(n)$ and committee sizes $k(n)$ can we achieve a size k committee with $> 2k/3$ honest parties with overwhelming probability (in a statistical security parameter)?*

We show, via a closely-followed probabilistic analysis, that for any g,k such that $g(n)^2 \cdot k(n) > 2\ln(2)\sigma \cdot n^2$ we can achieve a committee of size k with $> 2k/3$ honest parties with probability $\geq 1 - 2^{-\sigma}$ when selected from n parties with gap g (i.e. $t = n/3 - g(n)$).

At first sight, it may seem tempting to balance $g(n)$ and $k(n)$ by setting $g(n) = k(n) = \sqrt[3]{4\ln(2)(\sigma+1)n^2}$. Then, informally, our protocol would start with eliminating $O(g(n)) = O(n^{2/3})$ pairs of parties to achieve the $n^{2/3}$ gap, and then elect a sub-committee of size $O(n^{2/3})$ with two-thirds honest majority which can run the original protocol, thereby obtaining $O(n^{2/3})$ round overhead.

Recursive Committee Selection. Surprisingly, we reduce the round overhead even further by setting $g(n) = 2\sqrt{\ln(2)(\sigma+1+\log\log n)n}$ and $k(n) = n/2$. In our protocol, after eliminating $O(\sqrt{n})$ pairs of parties in $O(\sqrt{n})$ rounds, we select a $k(n) = n/2$ sized subcommittee which has two-thirds honest majority with probability $\geq 1 - 2^{-\sigma}$. Naively, running the protocol with committee of size $n/2$ would result in a round overhead of $\Omega(n)$. However, note that we are running a two-thirds honest majority protocol also after the committee selection, and so we can apply our solution **recursively** by creating a gap of $g(k(n)) = g(n/2) = 2\sqrt{\ln(2)(\sigma+1+\log\log(n/2))(n/2)}$ in $g(n/2)$ rounds and choosing committee of size $k(k(n)) = n/4$ and so on.

We briefly mention two technical hurdles when proving the security of this approach and how this affects the finer details of our solution. Recall that our main claim is that choosing a committee of size *exactly* $k(n)$ results in a committee with two-thirds honest majority with overwhelming probability. In our proof, we make use of the Chernoff bound. However, to apply the Chernoff bound in this case, we need each honest party to be chosen into the committee *independently* from the other parties. Therefore, we can only choose a subcommittee of *expected* size $k(n)$ where each party is chosen into the subcommittee with probability $k(n)/n$. We first show that with overwhelming probability in a statistical security parameter σ, the chosen subcommittee will be of size between $k(n)/2$ to $3k(n)/2$ when $k(n) > 12\ln(2)(\sigma+1)$. Then, we optimize this and show that by rejection-sampling the committee to fall into the bound of $[k(n)/2, 3k(n)/2]$ a small constant number of times (say, 8), we can even relax the inequality requirement further. Then, we claim by union-bounding all recursive committee selections of the protocol that when setting $k(n) = n/2$ then at most $1.5k(n) = 3n/4$ parties are chosen into the committee, thereby shrinking the number of remaining parties by at least 25% and resulting in a logarithmic number of recursive calls. This brings us to the second technical challenge: When observing the chosen $g(n)$ and $k(n)$, it can be seen that we do not tightly match the inequality of $g(n)^2 k(n) > 2\ln(2)\sigma \cdot n^2$. This is because in the recursive setting we need to union bound the error probability across all recursive calls. Therefore, we need to slightly decrease the error probability in each recursive call. As a result, in the recursive variant of the gap-amplification technique, an additional $\log\log n$ term is added to the gap.

To summarize, we improve our protocol for securely computing a circuit C to achieve $\leq g(n)$ round overhead, $O\left(|C|\cdot\mathsf{Comm}(\pi_{\mathsf{mult}})+g(n)\cdot n^2\cdot\log(|C|/(n\cdot g(n)))\right)$ overall communication (across all parties, together), while retaining the $O(|C|)$ computational overhead per party (independent of the number of parties).

Remark 1.3 (Improvement on Previous Works). We note that the proposed gap amplification technique is secure against a static adversary in a similar way to [DKMS12, DKM+17]. However, while their approaches work for $t<n(1/3-\epsilon)$ for some constant $\epsilon>0$, our approach works for any $t<n/3$, covering the whole range of two-thirds honest majority. We stress that amplifying the gap to achieve $t<n(1/3-\epsilon)$ is possible, but requires $\Omega(n)$ round overhead, defeating the original purpose of the technique. Nonetheless, we stress that like these works we are focused on *unconditional* security. Thus, any committee-based technique, such as choosing a committee of size $O(\log n)$ with only *honest* majority (i.e. $n=2t+1$) is irrelevant in our setting, as it relies on computational assumptions to achieve security against adaptive adversaries to realize a broadcast channel.

Remark 1.4 (Gap Amplification and Honest Majority). We note that the proposed gap amplification technique also works in the "almost" honest majority setting (i.e. $n=2t+1+h(n)$ for $h(n)=\omega(\sqrt{n})$), as eliminating a semi-corrupt pair of parties (i.e., with a single honest and a single corrupted party) increases the gap in that setting as well. This is discussed and shown in Section 6.4. We conjecture that a small number of cheater identifications (rather than pair eliminations) can be used to extend our result to the whole majority setting (i.e., $n=2t+1$). We leave the study of this for future work. Notice that even in this setting, the $O(\log n)$-sized committee-election approach with *honest* majority (i.e. $n=2t+1$) doesn't work, since it requires at least $n=2t+\epsilon\cdot n$ for some constant $\epsilon>0$.

1.5 Outline

In Section 2, we provide the necessary preliminaries. Section 3 presents the main protocol to compute arithmetic circuits and an ideal verification functionality that is used by the protocol. Then, we show how to realize this functionality in two ways. A first verification protocol is described in Section 4. This protocol is the basis for an improved version that is then presented in Section 5 which yields the result described in Theorem 1.1. Each of the verification protocols checks the correctness of a set of multiplication triples, generated by running a semi-honest multiplication protocol over shared inputs. Then, in Section 6 we present the gap amplification technique which allows us to obtain the result in Theorem 1.2.

2 Preliminaries

Notation. Let $P_1,\ldots,P_n$ be the n parties and let t be the number of corrupted parties. When two-thirds honest majority is assumed we have $n=3t+1$. We use $[n]$ to denote the set $\{1,\ldots,n\}$. We use $[\![x]\!]_t$ to denote a secret sharing of x with threshold t (as defined below). For a probability event E, we denote by $\Pr[E]$ the probability of E occurring and denote by $\bar{E}$ the complement of event E.

2.1 Probability Theory

Lemma 2.1 (Chernoff Bound). *Let $X_1,\ldots,X_n$ be a set of n independent and identically distributed (i.i.d.) random variables taking values in $\{0,1\}$. Let $X=\sum_{i=1}^{n}X_i$ and $\mu=\mathbb{E}[X]$. Then for any $\delta>0$, we have* $\Pr[X\geq(1+\delta)\mu]\leq e^{-\frac{\delta^2\mu}{2+\delta}}$

2.2 Background in Ring Theory

Let R be *any* finite ring. We only assume procedures for adding and multiplying ring elements, as well as sampling uniformly random elements. A set $A \subseteq R$ is called *exceptional* if, for all $x,y \in A$ with $x \neq y$, $x-y$ is invertible.[2]

For the rest of the paper, let $\mathbb{A}$ be any of the largest exceptional subsets of R, and let $\omega_R = |\mathbb{A}|$. We will need the following lemma in our protocol.

Lemma 2.2. *Let $a,b \in R$, with $a \neq 0$. Then* $\Pr_{x \xleftarrow{\$} \mathbb{A}}[x \cdot a + b = 0] \leq 1/\omega_R$.

Proof. Let $x,y \in \mathbb{A}$ such that $x \cdot a + b = 0$ and $y \cdot a + b = 0$, then $(x-y) \cdot a = 0$, since $x-y$ is invertible, then $a = 0$, which is a contradiction. This shows that there can be at most one $x \in \mathbb{A}$ that satisfies $x \cdot a + b = 0$, and therefore the probability of this event happening for a random sample in $\mathbb{A}$ is at most $1/|\mathbb{A}| = 1/\omega_R$. □

We observe that if R is a field then we may take $\mathbb{A} = R$, and therefore $\omega_R = |R|$. On the other hand, if R is the ring of integers modulo 2^k, then there are no exceptional sets of size 3 or more, so we may take $\mathbb{A} = \{0,1\}$, and hence $\omega_R = 2$.

2.3 Threshold Linear Secret Sharing Schemes

Definition 2.3. A t-out-of-n secret sharing scheme is a protocol for a *dealer* holding a secret value v and parties $P_1,...,P_n$ consisting of two interactive algorithms:

- $\mathsf{share}(v)$ which takes a secret v and outputs shares $[\![v]\!]_t = (v_1,...,v_n)$. The dealer runs $\mathsf{share}(v)$ and provides P_i with a share of the secret v_i.
- $\mathsf{reconstruct}([\![v]\!]_t^T, i)$, which takes the shares $v_j, j \in T \subseteq \{1,...,n\}$, and outputs v or $\perp$. A subset of users T run $\mathsf{reconstruct}([\![v]\!]_t^T, i)$ to reveal the secret to party P_i by sending their shares to P_i. When the secret is revealed to all $i \in [n]$ we simply write $\mathsf{reconstruct}([\![v]\!]_t^T)$. The event where the output is $\perp$ should be independent of the honest parties' shares.

The scheme must ensure that no subset of t shares provides any information on v, while $v = \mathsf{reconstruct}([\![v]\!]_t^T, i)$ for T only if $|T| \geq t+1$.

We say that a sharing is $\mathsf{consistent}$ if $\mathsf{reconstruct}([\![v]\!]_t^T, i) = \mathsf{reconstruct}([\![v]\!]_t^{T'}, i)$ for any two sets of honest parties $T,T' \subseteq \{1,...,n\}$, and $|T|, |T'| \geq t+1$.

In addition to the above, we define the following notations and procedures:

- $\mathsf{share}(v, [\![v]\!]^T)$ takes a secret v and a set of fixed shares $[\![v]\!]^T = \{v'_j\}_{j|P_j \in T}$ where $|T| \leq t$, and outputs $[\![v]\!]_t = (v_1,...,v_n)$, where $v_j = v'_j$ for each $j \in T$.
- $[\![x_i|_i]\!]_t$ is a consistent secret sharing of some value x where x_i is a share of P_i.

Multiplication. While a linear secret sharing scheme allows for local addition of shared values, it does not allow local multiplication. We assume that, given $[\![x]\!]_t$ and $[\![y]\!]_t$, the parties can locally compute $[\![x \cdot y]\!]_{2t}$ (thus interaction is required for reducing the threshold). We denote the operation of computing the product's sharing with threshold $2t$ by $[\![x]\!]_t \cdot [\![y]\!]_t$.

[2] In a finite non-commutative ring, a is invertible if there exists b such that $a \cdot b = b \cdot a = 1$.

Instantiations. Two popular secret sharing schemes that satisfy the above requirements are the Shamir secret sharing scheme [Sha79] and replicated secret sharing [ISN89]. The former is favoured when working with a large number of parties, since the share size remains constant, whereas in replicated secret sharing the share size grows exponentially with the number of parties. On the other hand, replicated secret sharing immediately works on any ring, whereas the Shamir scheme requires interpolation and thus is more suitable for working over fields.

2.4 MPC Security Definition

In this work, we consider full security against a malicious adversary and use the standard definition of security based on the ideal/real model paradigm [Can00, Gol04].

Additive Security. We build on the notion of security up to additive attacks [GIP+14, GIP15] as previously done in [LN17, CGH+18] to achieve maliciously secure honest majority MPC. In a nutshell, a protocol is secure up to additive attacks if it allows the adversary to only inject an *additive* error to the (otherwise correct) output of the computation. While this notion is defined for arbitrary computations, we limit our scope to $\mathcal{F}^{+}_{\mathsf{Mul}}$— the functionality that computes the multiplication over R with security up to additive attacks, defined in functionality 2.4.

Functionality 2.4. $\mathcal{F}^{+}_{\mathsf{Mul}}$- Secure multiplication up to an additive attack

Let $\mathcal{S}$ denote the ideal-world adversary controlling a set I of $t<n/3$ parties.

1. Upon receiving the shares of $[\![x]\!]_t^{P\setminus I}$ and $[\![y]\!]_t^{P\setminus I}$ held by the honest parties, the functionality reconstructs x and y and computes the corrupted parties' shares $[\![x]\!]_t^I$,$[\![y]\!]_t^I$.
2. The functionality sends $[\![x]\!]_t^I$ and $[\![y]\!]_t^I$ to $\mathcal{S}$.
3. Given an additive error value d and a set of desired output shares $\{\alpha_i\}_{i\in I}$ from $\mathcal{S}$ the functionality computes $z=x\cdot y+d$ and generates a random threshold-t sharing of z under the constraint that party P_i's share of $[\![z]\!]_t$ equals α_i.
4. The functionality sends the honest parties their shares in $[\![z]\!]_t$.

It has been shown that many existing semi-honest multiplication protocols in the honest majority setting are in fact secure up to an additive attack, albeit with minor modification to the protocol [LN17, CGH+18]. This includes the Damgard-Nielsen protocol [DN07] and its optimized versions [GLO+21], which are considered the state-of-the-art semi-honest protocols in this setting.

2.5 The Player Elimination Framework

Our protocol follows the general player elimination framework [HMP00]. According to this approach, the circuit is divided into $O(n)$ segments. The computation of each segment has three steps: (i) *Semi-honest computation:* the parties first evaluate the segment using a semi-honest protocol (which is private in the presence of malicious adversaries), (ii) *Verification*: the parties run a verification step to ensure correctness. If verification succeeds, the parties proceed to the

next segment. Otherwise, the parties find a *semi-corrupt* pair which is a pair of parties with the guarantee that at least one of them is corrupt, and proceed to the next step. (iii) *Recovery*: the parties remove this pair from the protocol and recompute the current segment. Note that the number of repetitions is bounded by $t = O(n)$, and so by dividing the circuit into $O(n)$ segments, we achieve that in the worst case, each gate is evaluated approximately once.

Our first contribution is designing a new protocol for (ii), i.e., a new verification protocol that works for any semi-honest protocol that is additively secure as defined above. Our second contribution changes the above framework by removing many parties together instead of just a single pair.

2.6 Ideal Functionalities

$\mathcal{F}_{\mathsf{coin}}$ - Generating Random Coins. Let $\mathcal{F}_{\mathsf{coin}}$ be an ideal functionality that hands the parties random elements in the ring. It is possible to generate an infinite number of elements by calling $\mathcal{F}_{\mathsf{coin}}$ once and using the obtained random element r as a seed to a pseudorandom generator. Alternatively, it is possible to derive the randomness in an information-theoretic way by taking $r, r^2, r^3, \ldots$. In any case, any fully secure realization of $\mathcal{F}_{\mathsf{coin}}$ will suffice.

$\mathcal{F}_{\mathsf{bc}}$ - Broadcast. We define a secure broadcast functionality which allows the parties to broadcast a message to all the other parties. Fortunately, the number of times this functionality is called will be sublinear in the size of the circuit and so any reasonable implementation will suffice. Note that in our setting of $t < n/3$ a broadcast channel can be realized over point-to-point channels with quadratic communication overhead [BGP92, CW89].

3 Securely Computing Any Circuit in the $\mathcal{F}_{\mathsf{vrfy}}$-Hybrid Model

We begin by defining an ideal verification functionality and describe the main protocol that makes use of this functionality to compute any circuit with full security, via the player elimination framework.

The Ideal Verification Functionality $\mathcal{F}_{\mathsf{vrfy}}$. Our protocol works in the hybrid model by calling the ideal functionality described in Functionality 3.1. We will present two ways to realize it in Section 4 and 5. The functionality receives the shares of many multiplication triples from the honest parties to check their correctness. The honest parties' shares suffice for computing all the secrets and determine whether cheating took place or not. Note that the functionality hands the adversary the additive errors and the corrupted parties' shares. These are anyway known to the adversary in our main protocol that calls $\mathcal{F}_{\mathsf{vrfy}}$, so nothing new is revealed to the adversary. The functionality outputs either accept (only if all triples are correct) or a semi-corrupt pair. A *semi-corrupt pair* is a pair of parties with the guarantee that at least one of them must be corrupt. Note that it allows $\mathcal{A}$ to cause the output to be a pair to remove even in the case where all triples are correct. This is required because, in our protocol that realizes $\mathcal{F}_{\mathsf{vrfy}}$, the adversary can make the verification fail even if no cheating took place and all triples are correct.

Functionality 3.1. $\mathcal{F}_{\mathsf{vrfy}}$ - Verifying correctness of a computation

The functionality is parametrized by a protocol π_{mult} to multiply shared secrets.
The functionality works with a set of parties $P_1, \ldots, P_n$ and an ideal world adversary $\mathcal{S}$ controlling a set I of t parties.

1. Upon receiving from the honest parties:
   - Their shares of $\{[\![x_k]\!]_t, [\![y_k]\!]_t, [\![z_k]\!]_t\}_{k=1}^m$
   - Their view (randomness, incoming/sent messages) in the execution of π_{mult} to compute each triple

   reconstruct $\{x_k, y_k, z_k\}_{k=1}^m$, compute the corrupted parties' shares and compute $d_k = z_k - x_k \cdot y_k$ for each $k \in [m]$.
2. Send $\mathcal{S}$ the corrupted parties' shares of $\{[\![x_k]\!]_t, [\![y_k]\!]_t, [\![z_k]\!]_t\}_{k=1}^m$ and $\{d_k\}_{k=1}^m$.
3. If $\forall k\colon\ d_k = 0$, then wait to receive $\mathsf{out} \in \{\mathsf{accept}, (j,k)\}$ from $\mathcal{S}$, such that j or k are in I, and then send out to the honest parties.
4. Otherwise, for each k such that $d_k \neq 0$, send the sent/received messages between honest and corrupted parties in the execution of π_{mult} to compute the kth triple to the ideal-world adversary.
   Then, wait to receive (j,k) from $\mathcal{S}$ such that j or k are in I, and then send (j,k) to the honest parties.

3.1 Some Building Blocks

$\mathsf{ss.check}([\![v_1]\!]_d, \ldots, [\![v_m]\!]_d, i)$ **- Consistency Check.** Let $\mathsf{ss.check}$ be a protocol that receives m sharings with threshold d (such that $d \leq 2t$) and an index i of the dealer party that dealt these sharings. The protocol outputs accept or a pair (i,k) with $k \in [n]$, so that P_i or P_k are guaranteed to be corrupted. This is done by taking a random linear combination of the sharings and opening the result. If the reconstruction does not succeed, the dealer broadcasts an index k of a party that published an incorrect share, and the parties output (i,k). If P_i is corrupted, then the output pair is semi-corrupt. Otherwise, P_i can identify the cheating party and so P_k is guaranteed to be corrupt. A formal description appears in Appendix A.

$\mathsf{vss.share}()$ **- Verifiable Secret Sharing.** A verifiable secret sharing is a protocol to share a secret, in a way that enables the parties to verify that the sharing is indeed consistent. This is achieved using Protocol A.1 for checking consistency. In particular, given a vector of secrets, the dealer runs the $\mathsf{share}()$ algorithm, hands the parties their shares and then the parties can run Protocol A.1.

3.2 The Main Protocol

Given $\mathcal{F}_{\mathsf{vrfy}}$, the protocol to compute any arithmetic circuit over a finite ring R works in a natural way. As explained before, the circuit is divided into segments, and each segment is computed sep-

arately. That is, the parties compute the segment using a semi-honest multiplication protocol, and then call $\mathcal{F}_{\mathsf{vrfy}}$ to verify the correctness of the computation. If the parties receive accept from $\mathcal{F}_{\mathsf{vrfy}}$, then they know that the secrets shared on the output layer of the current segment are correct, and so they can proceed to the next segment. Otherwise, they receive a semi-corrupt pair from $\mathcal{F}_{\mathsf{vrfy}}$ which is removed from the computation. This is carried out by updating the secret sharing of the inputs to the current segment using a recovery subprotocol. The segment is then recomputed with fewer parties.

Denote by $\mathsf{refresh}()$ a secure protocol that takes the sharing on the output wires of a given segment and reshares them with reduced threshold and across fewer parties. Specifically, the threshold is reduced by 1 and the number of parties is reduced by 2. Note that the security requirement that $t < n/3$ is preserved. Since both full protocol and the reshare procedure are standard, we defer their formal description and a formal proof of security to Appendices B and C.

Cost Analysis. Denote the semi-honest multiplication protocol by π_{mult} and denote the circuit size by $|C|$ (measured by the number of multiplication gates). The circuit is divided into $O(n)$ segments and so the number of calls to $\mathcal{F}_{\mathsf{vrfy}}$ is $O(n)$ and the input to $\mathcal{F}_{\mathsf{vrfy}}$ is of size $|C|/O(n)$. Denote by $C_{vrfy}(m)$ the total communication cost of $\mathcal{F}_{\mathsf{vrfy}}$, by $R_{vrfy}(m)$ its number of rounds and by $W_{vrfy}(m)$ the computational work per party, when the input to $\mathcal{F}_{\mathsf{vrfy}}$ is of size m. The communication cost of our protocol when all parties act honestly is therefore

$$|C|\cdot\mathsf{Comm}(\pi_{mult})+O(n\cdot C_{vrfy}(|C|/n)) \tag{1}$$

sent ring elements. Note that we ignore here the cost of the refresh protocol, assuming the width of the circuit is strictly smaller than its size. Also, note that some of the gates may be computed more than once if segments are recomputed. However, by choosing the number of segments to be $k \cdot n$ with $k \gg 1$, this additional cost can be ignored. Finally, note that for some rings, we may need to repeat the verification multiple times, and so the additive term is multiplied by a security parameter. Next, the overall number of rounds is

$$O(\mathsf{depth}(C)\cdot\mathsf{Round}(\pi_{\mathsf{mult}}))+O(n\cdot R_{vrfy}(|C|/n)). \tag{2}$$

Note however that the parties can start optimistically the next segment while running the verification protocol and revert if cheating was detected. Hence, if all parties act honestly, the verification does not incur any extra rounds.

Finally, the overall computational work per party is

$$O(|C|\cdot\mathsf{Comp}(\pi_{\mathsf{mult}}))+O(n\cdot W_{vrfy}(|C|/n)). \tag{3}$$

4 Securely Realizing $\mathcal{F}_{\mathsf{vrfy}}$ via Robust Recursive Search

In this section, we present our first verification protocol. The protocol takes as input parties' shares on the input and output wires of all multiplication gates, and outputs accept or identifies a semi-corrupt pair. We will see that if our protocol outputs accept, then the honest parties can be sure that no cheating took place during the semi-honest computation (except for a small probability).

4.1 Building Blocks

zeroShare() - Generating $[\![0]\!]_d$. In our protocol, the parties make use of a random sharing of 0 with threshold $d \leq 2t$. To achieve this, we use a similar protocol to the above: each party shares 0 to the other parties. Then, the parties run the consistency check (Protocol A.1) with a small modification: the parties output accept only if the reconstructed value is 0. Otherwise, the dealer outputs an index k of a cheating party and the parties output (i,k). Finally, the parties locally sum the n zero sharings and output the result.

"Mini" MPC to Find a Semi-Corrupt Pair. We define two ideal functionalities to locate a conflicting pair of parties in the computation of a single gate or when publishing a share of a single multiplication triple.

- $\mathcal{F}^1_{\mathsf{findPair}}$: This functionality receives from the honest parties their shares of a multiplication triple x,y,z and a description of a protocol that was used to compute $z = x \cdot y$. In addition, it receives the protocol transcript (that is, sent / received messages). The ideal functionality computes the corrupted parties' shares of the inputs and sends them and the messages exchanged between honest and corrupted parties to the ideal-world adversary.

  The functionality recomputes the messages that should have been sent by each corrupted party to an honest party and then:

  - If all messages sent from corrupted parties to honest parties are correct, then it waits to receive $\mathsf{out} \in \{\mathsf{accept}, (j,k)\}$ from $\mathcal{S}$, such that (j,k) is a semi-corrupt pair, and then sends out to the honest parties.
  - Otherwise, it waits to receive back a semi-corrupt pair from the adversary and hands it to the honest parties.

- $\mathcal{F}^2_{\mathsf{findPair}}$: This functionality receives shares of a single multiplication triple (x,y,z), a sharing of 0 with threshold $2t$ and a set of shares $(v_1,...,v_n)$ from the honest parties. The functionality computes the shares of the corrupted parties from the honest parties' shares. Then, for each $i \in [n]$ it checks that $v_i = z_i - x_i \cdot y_i + 0_i$ (where $z_i, y_i, x_i, 0_i$ are P_i's shares of $z,y,x,0$ respectively). The functionality sends the lowest index i for which the equation does not hold to the ideal-world adversary, to receive back an index k of another party. Finally, it sends (i,k) to the honest parties.

Both functionalities are invoked at most once in our verification protocol and over an input of a small constant size. Hence, any implementation will suffice.

4.2 The Protocol

Our protocol is formally described in Protocol 4.1. The protocol receives a set of multiplication triples (x_k, y_k, z_k) shared across the parties. The protocol first checks whether $u = \sum_{k=1}^{m} \alpha_k \cdot (z_k - x_k \cdot y_k) = 0$. If this holds, the parties output accept. Otherwise, they proceed to perform a binary search using the search procedure until locating a single triple (a,b,c) for which either the shares published by the parties are inconsistent or $c \neq a \cdot b$. In the first case, the parties call $\mathcal{F}^2_{\mathsf{findPair}}$, whereas in the second case they call $\mathcal{F}^1_{\mathsf{findPair}}$. As explained in the technical overview, for security, in each step the parties randomize the revealed shares by adding a sharing of 0 (produced by calling zeroShare()). In addition, to prevent the "slow-down" attack, the parties check in search() only one child of each

node and determine the next step according to its value. If the checked value is not 0, the parties proceed to the next step with the set of triples and published shares of the current node. Otherwise, they locally compute the shares on the other child and proceed to the next iteration with the set on that node.

Protocol 4.1. Verification Protocol

Input: The parties hold $\{[\![x_k]\!]_t,[\![y_k]\!]_t,[\![z_k]\!]_t\}_{k=1}^m$ and the transcript of the protocol to compute each $z_k=x_k\cdot y_k$.
The protocol:

1. The parties call $\mathcal{F}_{\mathsf{coin}}$ and $\mathsf{zeroShare}()$ to receive $\alpha_1,...,\alpha_m$ and $[\![0]\!]_{2t}$.
2. The parties locally compute $[\![u]\!]_{2t}=\sum_{k=1}^m\alpha_k\cdot([\![z_k]\!]_t-[\![x_k]\!]_t\cdot[\![y_k]\!]_t)+[\![0]\!]_{2t}$ (to perform the addition across thresholds, they first compute $[\![z]\!]_{2t}=[\![z]\!]_t\cdot[\![1]\!]_t$ where $[\![1]\!]_t$ is constant and public).
3. The parties run $\mathsf{reconstruct}([\![u]\!]_{2t})$ by broadcasting their shares to each other. Let u_i be the share broadcasted by P_i.
   - Case 1: $u=0$. In this case, the parties output accept.
   - Case 2: $u\neq 0$ or reconstruction fails, the parties call $\mathsf{search}(u_1,\dots,u_n,\alpha_1,...,\alpha_m,([\![x_1]\!]_t,[\![y_1]\!]_t,[\![z_1]\!]_t),...,([\![x_m]\!]_t,[\![y_m]\!]_t,[\![z_m]\!]_t),[\![0]\!]_{2t})$ to receive back $(t_1,...,t_n,[\![a]\!]_t,[\![b]\!]_t,[\![c]\!]_t,[\![0]\!]_{2t})$.
     - If $\mathsf{reconstruct}(t_1,...,t_n)\neq 0$, the parties call $\mathcal{F}^1_{\mathsf{findPair}}$ by sending $([\![a]\!]_t,[\![b]\!]_t,[\![c]\!]_t)$ and the transcript of the protocol (including correlated randomness used in the protocol if exists) to compute $[\![c]\!]_t$ to the functionality, and output whatever they receive back from the functionality (accept or (i,k)).
     - If $\mathsf{reconstruct}(t_1,...,t_n)=\perp$, the parties call $\mathcal{F}^2_{\mathsf{findPair}}$ by sending $(t_1,...,t_n,[\![a]\!]_t,[\![b]\!]_t,[\![c]\!]_t,[\![0]\!]_{2t})$ to the functionality, to receive back a pair (i,k). The parties output (i,k) and halt.

$\underline{\mathsf{search}(u_1,...,u_n,\alpha_1,...,\alpha_\ell,([\![x_1]\!]_t,[\![y_1]\!]_t,[\![z_1]\!]_t),...,([\![x_\ell]\!]_t,[\![y_\ell]\!]_t,[\![z_\ell]\!]_t),[\![0]\!]_{2t})}$:

1. The parties run $\mathsf{zeroShare}()$ to receive $[\![0_L]\!]_{2t}$.
2. The parties locally compute $[\![w]\!]_{2t}=\sum_{k=1}^{\ell/2}\alpha_k\cdot([\![z_k]\!]_t-[\![x_k]\!]_t\cdot[\![y_k]\!]_t)+[\![0_L]\!]_{2t}$.
3. The parties run $\mathsf{reconstruct}([\![w]\!]_{2t})$ by broadcasting their shares to each other. Let w_i be the share broadcasted by P_i.
   - Case 1: $w=0$. The parties compute $v_i=u_i-w_i$ for each $i\in[n]$ and $[\![0_R]\!]_{2t}=[\![0]\!]_{2t}-[\![0_L]\!]_{2t}$.
     - If $\ell=2$, return $(v_1,...,v_n,[\![x_\ell]\!]_t,[\![y_\ell]\!]_t,[\![z_\ell]\!]_t,[\![0_R]\!]_{2t})$.
     - If $\ell>2$, the parties call $\mathsf{search}(v_1,...,v_n,\alpha_{\ell/2+1},...,\alpha_\ell,([\![x_{\ell/2+1}]\!]_t,[\![y_{\ell/2+1}]\!]_t,[\![z_{\ell/2+1}]\!]_t),...,([\![x_\ell]\!]_t,[\![y_\ell]\!]_t,[\![z_\ell]\!]_t),[\![0_R]\!]_{2t})$ and return the result.
   - Case 2: $w\neq 0$ or w cannot be reconstructed.
     - If $\ell=2$, return $(w_1,...,w_n,[\![x_1]\!]_t,[\![y_1]\!]_t,[\![z_1]\!]_t,[\![0_L]\!]_{2t})$.
     - If $\ell>2$, the parties call $\mathsf{search}(w_1,...,w_n,\alpha_1,...,\alpha_{\ell/2},([\![x_1]\!]_t,[\![y_1]\!]_t,[\![z_1]\!]_t),...,([\![x_{\ell/2}]\!]_t,[\![y_{\ell/2}]\!]_t,[\![z_{\ell/2}]\!]_t),[\![0_L]\!]_{2t})$ and return the result.

Lemma 4.2. *Let R be a finite ring where size of the largest exceptional subset is ω_R (see Sec-*

tion 2.2). Assume for some $k \in [m]$ it holds that $z_k - x_k \cdot y_k \neq 0$. Then, the honest parties output accept *in Protocol 4.1 with probability of at most* $\frac{\log m}{\omega_R}$.

Proof. Assume $\exists k \in [m]\colon\ z_k - x_k \cdot y_k \neq 0$. Note that there are two events in our protocol where the parties output accept: (i) if $u = \sum_{k=1}^{m} \alpha_k \cdot (z_k - x_k \cdot y_k) = 0$ in the first round; (ii) the triple reached in the last round is correct. By Lemma 2.2, we have that $\Pr[u=0] \leq 1/\omega_R$. Now, the second event happens if in some step of the search, we have $w=0$ although k is in the verified set, which causes the parties to proceed to the next step of the recursion with a subset of correct triples. This event again can happen with probability $1/\omega_R$. By the union bound over all rounds, we have that the probability is at most $\frac{\log m}{\omega_R}$. □

The lemma above implies that for small ω_R we must work over a larger ring and/or repeat the protocol multiple times in order to obtain sufficiently small cheating probability. For example, if R is $\mathbb{Z}_{2^k}$, we have $\omega_R = 2$, and so to obtain cheating probability 2^{-s} for some $s \in \mathbb{N}$, the protocol must be executed over a larger ring R' such that $\frac{\log m}{\omega_{R'}} < 1$ and then be repeated t times such that $\left(\frac{\log m}{\omega_{R'}}\right)^t < 2^{-s}$.

Security. We prove that our protocol is secure in Appendix D.

Theorem 4.3. *Protocol 4.1 securely computes* $\mathcal{F}_{\mathsf{vrfy}}$ *with statistical security in the* $\mathcal{F}_{\mathsf{coin}}$*-hybrid model against malicious adversaries controlling up to* $t < n/3$ *parties.*

4.2.1 Removing $O(n)$ Term from the Communication Cost

In our protocol, the parties run the reconstruct procedure in each round. Naively, this requires each party to send its share to all other parties, implying $O(n^2)$ communication per round and so $O(\log(|C|) \cdot n^2)$ overall. To improve this, we can batch reconstructions in the following way. In the reconstruction, each party will send its share to P_1 who will reconstruct the secret and broadcast the computed secret to the other parties, or, alternatively will claim that the sharing is inconsistent. This immediately implies linear communication per reconstruction. However, P_1 may cheat. We observe that to deal with this threat, it suffices for the parties to publish the share of the last value in the search. Namely, if the triple $\bar{k}$ was located at the end of the search, the parties publish their randomized shares of $\alpha_{\bar{k}} \cdot (z_{\bar{k}} - x_{\bar{k}} \cdot y_{\bar{k}})$. If the sharing is inconsistent or does not reconstruct to 0, then the parties can proceed with calling $\mathcal{F}_{\mathsf{findPair}}$. Otherwise, if the shares are consistent and reconstruct to 0, then there must be a disagreement between some party P_k and P_1, as P_1 knows all the shares and claims that the shares do not define the value 0. Hence, P_1 can broadcast an index k of a party who sent an incorrect share and (i,k) is a new semi-corrupt pair. It follows that the communication complexity is now reduced to $O(\log|C| \cdot n) + O(n^2)$.

4.2.2 Cost Analysis

If all triples are correct, and all parties act honestly, then the protocol has a constant number of rounds and $O(n^2)$ communication cost. However, if cheating takes place, then in the worst case we have $O(\log(m))$ rounds and in each round the parties communicate to open a single secret. Hence, the number of rounds is $O(\log(m))$ and the communication cost is $O(\log(m) \cdot n)$ sent ring elements (using the optimization described above). Note that the computational cost is in any case

$O(m)$ ring operations per party, since the parties need to compute the linear combination of all m triples. We ignore here the cost of producing the zero-sharings in $\mathsf{zeroShare}()$, which is called at most $\log(m)$ times and is dominated by the above cost.

Plugging the costs into the Eq. (1), we have that communication cost of the main protocol when using this section's verification protocol is $|C|\cdot|\pi_{mult}|+O(n^3)$ sent elements when all parties act honestly and $|C|\cdot|\pi_{mult}|+O(n^2\cdot\log(|C|/n))$ in the worst case. Similarly, by Eq. (2), we have that the number of rounds of the main protocol is $O(\mathsf{depth}(C))+O(n)$ in case no cheating took place and $O(\mathsf{depth}(C))+O(n\cdot\log(|C|/n))$ in the worst case. Finally, using Eq. (3) we obtain that the total computational work is $O(|C|)+O(n\cdot|C|/n)=O(|C|)$ arithmetic operations.

5 Realizing $\mathcal{F}_{\mathsf{vrfy}}$ with a Constant Number of Rounds via Recursive Search and DZKP

The framework presented so far achieves the desired overhead in terms of communication and computational overhead. On the other hand, the number of rounds is $O(n\cdot\log(|C|/n))$. Being logarithmic in the circuit's size is good when considering communication, since the communication of the semi-honest protocol typically grows linearly with the circuit's size. However, the number of rounds depends on the circuit's *depth* rather than its size. Hence, adding $O(\log(|C|))$ number of rounds to the semi-honest protocol can incur a significant overhead, e.g., when the circuit is large and shallow. In this section, we present an improved verification protocol that has a constant number of rounds, without giving up the communication and computational overhead obtained in the previous section. We combine the protocol from Section 4 with a verification based on sublinear distributed zero-knowledge proofs (DZKP) to obtain the best of both worlds.

5.1 Additional Building Blocks

We define two ideal functionalities that are used in our protocol. Both can be seen as a generalization of $\mathcal{F}^1_{\mathsf{findPair}}$ and $\mathcal{F}^2_{\mathsf{findPair}}$ defined in Section 4. However, unlike the former functionalities which take a small constant-size input, these are defined over an input of arbitrary size. Hence, we cannot assume that any realization has constant cost. Instead, we need to show concrete protocols to realize these functionalities and evaluate their complexity. We also define two additional subprotocols $\mathsf{ss.check}$ and $\mathsf{ss.convert}$ that are used in the realization of these functionalities.

$\mathcal{F}_{\mathsf{checkShare}}$ - Proving Correctness of a Published Share. The functionality is defined in Functionality 5.1. It is invoked when a set of published shares $t_1,\ldots,t_n$ are not consistent, and each share t_i should be a result of computing $\sum_{k=1}^{\ell}\alpha_k\cdot(c_{k,i}-a_{k,i}\cdot b_{k,i})+0_i$. Hence, the honest parties give their shares of all these inputs to the functionality, allowing it to compute the corrupted parties' shares and find the cheater. The output is a semi-corrupt pair (i,k) where i is an index of one of the cheating parties.

Functionality 5.1. $\mathcal{F}_{\mathsf{checkShare}}$ - Verifying correctness of a published share

The functionality works with a set of parties $P_1,\ldots,P_n$ and an ideal world adversary $\mathcal{S}$ controlling t parties.

Upon receiving $(t_1,...,t_n,\{\alpha_k,[\![a_k]\!]_t,[\![b_k]\!]_t,[\![c_k]\!]_t\}_{k=1}^{\ell},[\![0]\!]_{2t})$ from the honest parties, the ideal functionality works by the following instructions:

1. Compute for each corrupted party P_i, the shares $a_{k,i},b_{k,i},c_{k,i}$ for each $k\in[\ell]$ and 0_i and sends them together with $t_1,...,t_n$ and $\alpha_1,...,\alpha_\ell$ to $\mathcal{S}$.
2. Check for each $i\in[n]$ that $t_i=\sum_{k=1}^{\ell}\alpha_k\cdot(c_{k,i}-a_{k,i}\cdot b_{k,i})+0_i$.
   Let I be the set of parties for which the equation does not hold.
3. Receive a pair (i,k) from $\mathcal{S}$ where $i\in I$ and hand the pair to the honest parties.

$\mathcal{F}_{\mathsf{checkTranscript}}$ **- Checking Semi-Honest Transcript.** The functionality is defined in Functionality 5.2.

Functionality 5.2. $\mathcal{F}_{\mathsf{checkTranscript}}$ **- Verify correctness of a multiplication protocol**

The functionality is parametrized by a multiplication protocol π_{mult}.
The functionality works with a set of parties $P_1,\dots,P_n$ and an ideal world adversary $\mathcal{S}$ controlling a set I of t parties.
Upon receiving from the honest parties: (i) their shares of $\{[\![a_k]\!]_t,[\![b_k]\!]_t,[\![c_k]\!]_t\}_{k=1}^{\ell}$; (ii) their view (i.e., randomness and sent/received messages) in the execution of π_{mult} to compute each $[\![c_k]\!]_t$, the ideal functionality works by the following:

1. Compute for each corrupted party P_i, the shares $a_{k,i},b_{k,i},c_{k,i}$ for each $k\in[\ell]$ and sends them together with all messages exchanged between corrupted and honest parties to $\mathcal{S}$.
2. Check for each $k\in[\ell]$ and each $i\in I$ that the message sent by P_i in the computation of the kth triple is correct.
   - If all messages were correct, then wait to receive $\mathsf{out}\in\{\mathsf{accept},(j,t)\}$ from $\mathcal{S}$, such that j or t are in I, and then send out to the honest parties.
   - If cheating was found, wait to receive (j,t) from $\mathcal{S}$ such that j or t are in I and then send (j,t) to the honest parties.

$(j,k)\leftarrow\mathsf{ss.check}([\![x]\!]_d)$ **- Resolving Inconsistency.** Let $\mathsf{ss.check}([\![x]\!]_d)$ be a protocol that takes a sharing $[\![x]\!]_d$ (with threshold $t<d\leq 2t$) that was found to be inconsistent and outputs a semi-corrupt pair. The input of the parties includes all shares of x as published by the parties (since the inconsistency was discovered when reconstructing x). We use a protocol from [GSZ20, DEN24]. For completeness, the formal description and analysis can be found in Appendix E. The protocol assumes that $[\![x]\!]_d$ can be decomposed into $[\![x^i]\!]_d$ such that $[\![x]\!]_d=\sum_{i=1}^{n}\gamma_i\cdot[\![x^i]\!]_d$, where $[\![x^i]\!]_d$ was shared by party P_i and γ_i is public. Indeed, this is the case whenever we call this protocol.

$[\![r_i|_i]\!]_t\leftarrow\mathsf{ss.convert}([\![r_i|_i]\!]_{2t},i)$ **- Converting to a Different Threshold.** This protocol takes a sharing with threshold $2t$ and converts it to a sharing with a reduced threshold t. More specifically,

the input is a sharing with threshold $2t$ where the share of P_i is r_i, and the parties convert it to a sharing of a random value with threshold t, where the sharing of P_i remains r_i. The protocol may fail, and in this case it outputs a semi-corrupt pair. In addition, a malicious P_i who wishes to cheat and outputs a sharing of a different share may succeed but with a small probability. If the cheating probability is not sufficiently small, the parties can repeat the protocol multiple times. In any case, we call this protocol only once, and so its cost is amortized away. The formal description appears in Appendix F .

Proving Degree-2 Relations. We also define an ideal functionality that checks correctness of a degree-2 computation performed by some party P_i. In particular, the parties verify that $c_i - \sum_{k=1}^{m} a_{k,i} \cdot b_{k,i} = 0$ where c_i, $a_{k,i}$ and $b_{k,i}$ are party P_i's shares of c,a_k and b_k. If the equation does not hold, the functionality outputs a semi-corrupt pair. Note that as before, even if the equation holds, the functionality allows the ideal-world adversary to cause the output to be a semi-corrupt pair.

Functionality 5.3. $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ - proving degree-2 statements

The functionality works with a set of parties $P_1, \ldots, P_n$ and an ideal world adversary $\mathcal{S}$ controlling a set I of t parties.

Upon receiving $[\![c]\!]_t$, $[\![a_1]\!]_t, \ldots, [\![a_\ell]\!]_t$, $[\![b_1]\!]_t, \ldots, [\![b_\ell]\!]_t$ and an index i from the honest parties, the ideal functionality computes P_i's shares c_i, $a_{1,i}$, ..., $a_{\ell,i}$, $b_{1,i}$, ..., $b_{\ell,i}$ and checks that $c_i - \sum_{k=1}^{\ell} a_{k,i} \cdot b_{k,i} = 0$.

- If the equation holds, then wait to receive $\mathsf{out} \in \{\mathsf{accept}, (i,k)\}$ from $\mathcal{S}$, such that i or k are in I, and then send out to the honest parties.
- If the equation does not hold, wait to receive (i,k) from $\mathcal{S}$ and then send (i,k) to the honest parties.

We rely on the following result:

Theorem 5.4 ([DEN24])**.** *There exists a protocol that securely computes $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ with statistical security where the overall communication cost is $O(n \cdot \log(\ell))$, the computational work per party is $O(\ell)$ operations and the number of rounds is $O(1)$ in the random oracle model.*

The protocol from [DEN24] makes use of distributed zero-knowledge proofs over secret shared data (DZKP) which rely on the abstract primitive of zero-knowledge fully linear interactive oracle proofs (zk-FLIOP) [BBC+19]. Several previous works have used this primitive to prove correctness in the context of MPC [BGIN19, BGIN20, BGIN21, BGIN22]. However, the authors of [DEN24] were the first to present a protocol with the above complexity measures for the setting where a party wishes to prove correctness of a computation over its shares without knowing the other parties' shares, as in our setting.

5.1.1 Securely Computing $\mathcal{F}_{\mathsf{checkShare}}$

In this section, we present a protocol to check the correctness of shares published by the parties. Namely, given a set of shares t_1,...,t_n, the protocol checks that each share results from computing

$\sum_{k=1}^{\ell}\alpha_k\cdot(\llbracket z_k\rrbracket_t-\llbracket a_k\rrbracket_t\cdot\llbracket b_k\rrbracket_t)+\llbracket 0\rrbracket_{2t}$. Since the shares are inconsistent, we expect that the protocol will find an index i for which t_i was not computed correctly. The protocol is described in Protocol 5.5. Given the functionality $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ the protocol is simple. Recall that $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ receives sharings of c, $a_1,...,a_\ell$ and $b_1,...,b_\ell$ with threshold t and checks that P_i's share in c equals the inner product of its shares of $a_1,...,a_\ell$ and $b_1,...,b_\ell$. Hence, the protocol only needs to bring the input to the right form before sending it to $\mathcal{F}_{\mathsf{Deg2Vrfy}}$. This is done by using the ss.convert protocol to convert $\llbracket 0\rrbracket_{2t}$ to threshold t and summing the resulting threshold t sharings.

To see why the protocol is correct, note that in the protocol, $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ is called to check that $c'_i-\sum_{k=1}^{\ell}a'_{k,i}\cdot b_{k,i}=0$. Observe that $a'_{k,i}=\alpha_k\cdot a_{k,i}$ and $c'_i=t_i-\sum_{k=1}^{\ell}\alpha_k\cdot c_{k,i}+r_i$ where r_i is P_i's share in $\llbracket 0\rrbracket_{2t}$. Hence,

$$c'_i-\sum_{k=1}^{\ell}a'_{k,i}\cdot b_{k,i}=t_i-\sum_{k=1}^{\ell}\alpha_k\cdot c_{k,i}-r_i-\sum_{k=1}^{\ell}\alpha_k\cdot(a_{k,i}\cdot b_{k,i})=t_i-\sum_{k=1}^{\ell}\alpha_k\cdot(c_{k,i}-a_{k,i}\cdot b_{k,i})-r_i$$

which is exactly the expression for which we wish to check equality to 0.

Protocol 5.5. Check correctness of a public share

- **Input:** $t_1,...,t_n,\{\alpha_k,\llbracket a_k\rrbracket_t,\llbracket b_k\rrbracket_t,\llbracket c_k\rrbracket_t\}_{k=1}^{\ell},\llbracket 0\rrbracket_{2t}$.
  Denote the shares of P_i for each k by $a_{k,i},b_{k,i},c_{k,i}$ and the share of 0 by r_i.
- **The protocol:**
  1. For $i=1$ to n:
     (a) The parties call $\mathsf{ss.convert}(\llbracket 0\rrbracket_{2t},i)$ to receive $\llbracket r_i|_i\rrbracket_t$.
     (b) The parties locally compute $\llbracket c'\rrbracket_t=\llbracket t_i|_i\rrbracket_t-\sum_{k=1}^{\ell}\alpha_k\cdot\llbracket c_k\rrbracket_t-\llbracket r_i|_i\rrbracket_t$
     where $\llbracket t_i|_i\rrbracket_t$ is a public sharing defined by running $\mathsf{share}(0,\{v'_j\}_{j\in T})$ such that $T=\{j_1,...,j_{t-1},i\}$ is a fixed set of size t and $v'_{j_1}=...=v'_{j_{t-1}}=0$, $v'_i=t_i$.
     (c) The parties call $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ over $\llbracket c'\rrbracket_t,\llbracket a'_1\rrbracket_t,...,\llbracket a_\ell\rrbracket_t,\llbracket b_1\rrbracket_t,...,\llbracket b_\ell\rrbracket_t$ and i, where $\llbracket a'_k\rrbracket_t=\alpha_k\cdot\llbracket a_k\rrbracket_t$ for each $k\in[\ell]$.
     If the parties receive a pair (i,k), the parties output the pair and halt.

Proposition 5.6. *Protocol 5.5 securely computes $\mathcal{F}_{\mathsf{checkShare}}$ with statistical security in the $\mathcal{F}_{\mathsf{Deg2Vrfy}}$-hybrid model in the presence of malicious adversaries controlling $t<n/3$ parties.*

Proof. We describe an ideal-world simulator $\mathcal{S}$ that interacts with a real world adversary $\mathcal{A}$ as follows. The simulator knows the corrupted parties shares of a_k, b_k, c_k and 0 and the public values $t_1,...,t_n$ and $\alpha_1,...,\alpha_\ell$. Hence, $\mathcal{S}$ knows which t_i are incorrect. The simulator $\mathcal{S}$ runs the simulator for ss.convert described in Section 5.1. Then, it simulates $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ interacting with $\mathcal{A}$. Note that we know that the checked equation is correct for honest parties. In addition, $\mathcal{S}$ knows for each corrupted party if the checked equation holds or not. Hence, it can perfectly simulate the role of $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ for each $i\in[n]$ by following the instructions of the functionality. It follows that the simulation is identically distributed to a real world execution. The only difference is when the simulator for the conversion protocol outputs fail, which is why we only achieve statistical security. This concludes the proof. □

5.1.2 Securely Computing $\mathcal{F}_{\mathsf{checkTranscript}}$

We next show how to realize the ideal functionality $\mathcal{F}_{\mathsf{checkTranscript}}$, which searches for cheaters in the semi-honest protocol used to compute ℓ multiplication triples. The exact protocol depends, of course, on the multiplication protocol used to compute the circuit. As in recent works, we describe a protocol when the parties run the state-of-the-art Damgard-Nielsen semi-honest multiplication protocol [DN07]. We denote this protocol by $\pi_{\mathsf{checkTranscript}}$, and its formal description is given in Protocol 5.7.

The DN Protocol [DN07]. The popular DN protocol works as follows. In an offline phase, the parties generate a double-sharing $(\llbracket r \rrbracket_t, \llbracket r \rrbracket_{2t})$ for a random secret r. In the online phase, the parties multiply $\llbracket x \rrbracket_t$ and $\llbracket y \rrbracket_t$ in two rounds of interaction:

1. The parties locally compute $\llbracket x \cdot y - r \rrbracket_{2t} = \llbracket x \rrbracket_t \cdot \llbracket y \rrbracket_t - \llbracket r \rrbracket_{2t}$ and send their shares to P_1.
2. P_1 reconstructs $x \cdot y - r$ and sends it back to the parties, who locally compute $\llbracket x \cdot y \rrbracket_t = \llbracket r \rrbracket_t + (x \cdot y - r)$.

As observed by [GLO+21], it is possible to reduce communication in the second round by having P_1 send $x \cdot y - r$ to $n - t$ parties only. The parties can then define a secret sharing $\llbracket x \cdot y - r \rrbracket_t$ by setting the share of the remaining parties to be 0. Then, the parties locally compute $\llbracket r \rrbracket_t + \llbracket x \cdot y - r \rrbracket_t$ and obtain the result. In [GLO+21], another variant was proposed that batches the two rounds over two layers of the circuit (but with slightly more communication), thereby obtaining one round per gate. Our construction works easily with that variant as well.

Protecting against the Double-Dipping Attack. As our protocol follows the paradigm of verifying correctness only at the end of the computation, it is crucial that the multiplication protocol will provide privacy even in the presence of malicious adversaries. Perhaps surprisingly, it was shown that privacy does not hold for the textbook DN protocol in the strong honest majority setting, i.e., when $2t + 1 < n$, as in our case. The attack which was first introduced by Goyal et al. [GLS19] is known as the "double-dipping" attack, as it is carried out over two layers of the circuit. The attack is executed by a malicious P_1 and relies on the fact that once P_1 receives $2t$ sharings, it can compute $x \cdot y - r$ and then *compute the remaining* $n - 2t - 1$ *shares*. Now, assume that P_n is honest and a malicious P_1 sends an incorrect message only to P_n. As a result, in the next layer, party P_n will hand P_1 an incorrect share. However, due to the redundancy in the secret sharing, P_1 will be able to compute the share P_n should have sent from other $2t+1$ correct shares it sees. Then, P_1 can compute the difference between the actual message received from P_n and the message it should have sent. From this difference, P_1 is able to extract private information. We remark that the attack works even if we decide that only $2t$ parties will communicate their shares to P_1, as some of the remaining parties may collude with P_1 and hand him their shares on the wires.

There are several solutions in the literature to this attack. Goyal et al. [GLS19] suggested to share the mask using an additive secret sharing instead of using $2t$-sharing. This solves the attack as now there is no redundancy in the secret sharing anymore. However, it increases the communication of the protocol, since now all $n-1$ parties need to send messages to P_1 instead of just $2t$. Moreover, if we use this solution in our protocol, we will lose the robustness of $\llbracket r \rrbracket_{2t}$ which is relied upon in our verification protocol. Furukawa and Lindell [FL19] solved the attack by running a constant-cost consistency check between each two layers. This solves the attack since any inconsistency from the

side of P_1 is identified before the computation of the next layer. However, this adds an extra round of communication to each layer. Recently, Escudero et al. [DEN24] used an approach where $[\![r]\!]_t$ is shared only across $2t+1$ parties, whereas $[\![r]\!]_{2t}$ is shared across all parties. This prevents the attack since only $2t+1$ parties know the sharing on the input wires of each gate, and so there is no redundancy in the message to P_1. Unfortunately, we cannot use this solution in our protocol as well, since we require that all parties hold the state on each wire (so they can perform the check that $\sum_{k=1}^{m} \alpha_k \cdot (z_k - x_k \cdot y_k) = 0$). As can be seen from the discussion, the only solution that fits our framework is the Furukawa-Lindell solution where the parties check between each layer that P_1 sent the same message. This can be done efficiently by publishing a collision-resistance hash of all messages sent by P_1 in the previous layer.

The Protocol of $\pi_{\mathsf{checkTranscript}}$. We are now ready to present the protocol to verify the correctness of messages in ℓ executions of the multiplication protocols. Our protocol follows the ideas of [DEN24] with some small modifications. The main idea is that the parties first agree on a compressed transcript of the ℓ executions by taking a random linear combination of each message (i.e., messages from P_i to P_1 and from P_1 to the other parties) and publishing them. The observation made by [DEN24] is that there is no need to verify the correctness of P_1's message since it is the sum of all first-round messages. Instead, once there is agreement on P_1's message and on the messages sent to P_1, the parties can compute P_1's implicit message in the first round, i.e., P_1's share of $x \cdot y - r$. Then, the parties verify the correctness of all first-round messages. This suffices since if all messages to P_1 are correct but P_1 sent an incorrect message, then this is equivalent to computing its message by adding an incorrect share. Hence, the parties will detect it by verifying P_1's implicit first-round message. The first-round messages are verified by calling $\mathcal{F}_{\mathsf{Deg2Vrfy}}$. Specifically, each message is a share computed by taking $[\![x_k]\!]_t \cdot [\![y_k]\!]_t - [\![r_k]\!]_{2t}$, which is a degree-2 computation. The parties take the random linear combination $\sum_{k=1}^{\ell} \gamma_k \cdot ([\![x_k]\!]_t \cdot [\![y_k]\!]_t - [\![r_k]\!]_{2t})$ which can be written as $\sum_{k=1}^{\ell} [\![x'_k]\!]_t \cdot [\![y_k]\!]_t - [\![r]\!]_{2t}$ such that $x'_k = \gamma_k \cdot x_k$ and $r = \sum_{k=1}^{\ell} \gamma_k \cdot r_k$. The parties then use our conversion protocol to obtain a sharing $[\![r_i|_i]\!]_t$, i.e., a sharing with threshold t, where P_i's share is the same as in $[\![r]\!]_{2t}$. Now that we have an expression of the correct form, the parties can call $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ to check that all parties computed their published share correctly.

Protocol 5.7. $\pi_{\mathsf{checkTranscript}}$**: Computing** $\mathcal{F}_{\mathsf{checkTranscript}}$

Let ℓ be the number of multiplication operations to verify. Let $x_{k,i}, y_{k,i}$ and $r_{k,i}$ be the shares held by P_i in $[\![x_k]\!]_t, [\![y_k]\!]_t$ and $[\![r_k]\!]_{2t}$. Let $\mathsf{msg}_{i,k}$ be the message sent from each P_i to P_1 and let $\mathsf{msg}_{1,k}$ be the message sent by P_1, in the computation of the kth gate.

The protocol: Part I: reach an agreement on a compressed transcript:

1. The parties call $\mathcal{F}_{\mathsf{coin}}$ to receive $\gamma_1, \ldots, \gamma_\ell$.
2. Each party P_i $(i \neq 1)$ broadcasts (via $\mathcal{F}_{\mathsf{bc}}$) $\mathsf{msg}^i = \sum_{k=1}^{\ell} \gamma_k \cdot \mathsf{msg}_{i,k}$ and $\mathsf{msg}^{rnd\ 2} = \sum_{k=1}^{\ell} \gamma_k \cdot \mathsf{msg}_{1,k}$ to the other parties.
   In parallel, P_1 broadcasts (via $\mathcal{F}_{\mathsf{bc}}$) msg^i and $\mathsf{msg}^{rnd\ 2}$ computed in the same way for each $i \neq 1$.
3. If there is any contradiction between a message published by some P_i $(i \neq 1)$ and P_1, then the parties output $(1, i)$ and halt. Otherwise, they proceed to the next step.

Part II: verify correctness of compressed messages:

Let $\mathsf{msg}^1 = \mathsf{msg}^{rnd\ 2} - \sum_{i=2}^{n} \mathsf{msg}^i$.
The parties work as follows:

1. The parties locally compute $[\![r]\!]_{2t} = \sum_{k=1}^{\ell} \gamma_k \cdot [\![r_k]\!]_{2t}$.
2. For each $i \in [n]$, the parties run $[\![r_i|_i]\!]_t = \mathsf{ss.convert}([\![r]\!]_{2t}, i)$.
3. Set $[\![x'_1]\!]_t, ..., [\![x'_\ell]\!]_t = (\gamma_1 \cdot [\![x_1]\!]_t, ..., \gamma_\ell \cdot [\![x_\ell]\!]_t)$, $[\![y_1]\!]_t, ..., [\![y_\ell]\!]_t = ([\![y_1]\!]_t, ..., [\![y_\ell]\!]_t)$ and $[\![z^i]\!]_t = [\![\mathsf{msg}_i|_i]\!]_t - [\![r_i|_i]\!]_t$, where $[\![\mathsf{msg}_i|_i]\!]_t$ is defined by running $\mathsf{share}(0, \{v_j\}_{j \in T})$ where $T = \{j_1, ..., j_{t-1}, i\}$, for some fixed set $j_1, ..., j_{t-1}$ and $v_i = \mathsf{msg}_i$ and $v_{j_1} = \cdots = v_{j_{t-1}} = 0$.
4. For each $i \in [n]$, the parties call $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ over $[\![z^i]\!]_t$, $[\![x'_1]\!]_t, ..., [\![x'_\ell]\!]_t$, $[\![y_1]\!]_t, ..., [\![y_\ell]\!]_t$ and i.
   If the parties receive a pair (i,k), the parties output the pair and halt.
5. Otherwise, the parties output accept

Proposition 5.8. *$\pi_{\mathsf{checkTranscript}}$ (Protocol 5.7) securely computes $\mathcal{F}_{\mathsf{checkTranscript}}$ with statistical security in the $(\mathcal{F}_{\mathsf{coin}}, \mathcal{F}_{\mathsf{Deg2Vrfy}})$-hybrid model in the presence of malicious adversaries controlling $t < n/3$ parties.*

Proof. Let $\mathcal{S}$ be the ideal-world adversary and let $\mathcal{A}$ be the real-world adversary. The simulator $\mathcal{S}$ receives from the trusted party computing $\mathcal{F}_{\mathsf{checkTranscript}}$ the shares of the corrupted parties and the sent/received messages in the semi-honest protocol between honest and corrupted parties. We distinguish between two cases:

- *Case 1: P_1 is honest.* In part 1, $\mathcal{S}$ first plays the role of $\mathcal{F}_{\mathsf{coin}}$, handing the random coins to $\mathcal{A}$. Since $\mathcal{S}$ knows the messages sent to P_1 from corrupted parties and the message sent by P_1 in the second round, it can perfectly simulate the broadcast of $\mathsf{msg}^{rnd\ 2}$ and msg^i for each corrupted party P_i. For the message sent from an honest P_i to P_1, the simulator chooses random messages that, together with the shares sent by the corrupted parties, sum to P_1's message and publishes them.

  In part 2, $\mathcal{S}$ runs the simulator for $\mathsf{ss.convert}$ and then plays the role of $\mathcal{F}_{\mathsf{Deg2Vrfy}}$. For each honest P_i, the simulator knows that the output of $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ is accept. In this case, it waits for $\mathcal{A}$ to decide whether the output will be accept or a semi-corrupt pair chosen by $\mathcal{A}$. For each corrupt P_i, since $\mathcal{S}$ knows all input of P_i, it can recompute its message and determine whether the message is correct or not. Hence, once again it can simulate $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ perfectly.

- *Case 2: P_1 is corrupted.* In this case, $\mathcal{S}$ knows the entire transcript of the protocol (since all messages are exchanged with P_1). Hence, it can perfectly simulate part 1. In part 2, it works exactly as in the first case.

Overall, the only difference between an honest execution and the simulation is the event where the simulator of $\mathsf{ss.convert}$ outputs fail. However, this happens with small probability and thus the protocol is statistically secure as stated in the proposition. This concludes the proof. □

5.2 The Protocol

We are now ready to present our verification protocol. The formal description is given in Protocol 5.9. We present the formal description of the improved verification protocol from Section 5.2. The main difference compared to protocol 4.1 is that the recursive search stops once a set of size $\frac{m}{n}$ is

reached. Then, the parties call either $\mathcal{F}_{\mathsf{checkShare}}$ or $\mathcal{F}_{\mathsf{checkTranscript}}$ on the obtained set. The protocol takes as input a public parameter N that determines the folding factor of the search. In each round, the parties split the given set into N equally sized subsets and verify each of them separately by reconstructing the random linear combination and checking equality to 0. As in the previous section, we need to take into account the "slowdown" attack where reconstruction fails due to inconsistency, but in the next round all N subsets are successfully verified. To avoid this event, the protocol verifies $N-1$ subsets and if all were verified to be correct, the search proceeds automatically with the Nth subset to the next step of the recursion. For this to work, the parties compute the inputs to the next step deterministically from the inputs to the current step and the $N-1$ checks performed in the current step. In particular, say we entered the current step with shares $u_1,...,u_n$ for a random linear combination of ℓ triples, and obtained shares $w_{j,1},...,w_{j,n}$ for the reconstruction of the jth random linear combination (of a subset of size ℓ/N) for $j=1$ to $N-1$. Then, the share of P_i for the linear combination of the unopened Nth subset is simply $u_i-\sum_{j=1}^{N-1} w_{j,i}$. This means that if $u_1,...,u_n$ are inconsistent or reconstruct to a value $\neq 0$, and for each $j\in\{1,...,N-1\}$ the shares $w_{j,1},...,w_{j,n}$ reconstruct to 0, then $w_{N,1},...,w_{N,n}$ will inherit the property of $u_1,...,u_n$. Hence, we can indeed make the recursive step over the shares $w_{N,1},...,w_{N,n}$, and guarantee that the final step will yield a subset that is either incorrect or that the shares of the random linear combination are inconsistent. In the former case, the parties will call $\mathcal{F}_{\mathsf{checkTranscript}}$, whereas in the latter case, they will call $\mathcal{F}_{\mathsf{checkShare}}$. The formal protocols for $\mathcal{F}_{\mathsf{checkShare}}$ and $\mathcal{F}_{\mathsf{checkTranscript}}$ are given by Protocol 5.5 and $\pi_{\mathsf{checkTranscript}}$ (Protocol 5.7), respectively, and their security is analyzed in Sections 5.1.1 and 5.1.2, respectively.

Protocol 5.9. Verification Protocol: Improved Version

- **Public parameters:** A folding factor N.
- **Input**: The parties hold $\{[\![x_k]\!]_t,[\![y_k]\!]_t,[\![z_k]\!]_t\}_{k=1}^m$ and the transcript of the protocol to compute each $z_k=x_k\cdot y_k$.
- **The protocol**:
  1. The parties call $\mathcal{F}_{\mathsf{coin}}$ to receive $\alpha_1,...,\alpha_m$.
  2. The parties run $\mathsf{zeroShare}()$ to receive $[\![0]\!]_{2t}$.
  3. The parties locally compute $[\![u]\!]_{2t}=\sum_{k=1}^m \alpha_k\cdot([\![z_k]\!]_t-[\![x_k]\!]_t\cdot[\![y_k]\!]_t)+[\![0]\!]_{2t}$ (to perform the addition across thresholds, they first compute $[\![z]\!]_{2t}=[\![z]\!]_t\cdot[\![1]\!]_t$ where $[\![1]\!]_t$ is constant and public).
  4. The parties run $\mathsf{reconstruct}([\![u]\!]_{2t})$ by broadcasting their shares to each other. Let u_i be the share broadcasted by P_i.
     - Case 1: $u=0$. In this case, the parties output accept.
     - Case 2: $u\neq 0$ or reconstruction fails, the parties call $\mathsf{search}(u_1,\ldots,u_n,\alpha_1,...,\alpha_m,([\![x_1]\!]_t,[\![y_1]\!]_t,[\![z_1]\!]_t),...,([\![x_m]\!]_t,[\![y_m]\!]_t,[\![z_m]\!]_t),[\![0]\!]_{2t})$ to receive back $(t_1,...,t_n,\{\alpha_k,[\![a_k]\!]_t,[\![b_k]\!]_t,[\![c_k]\!]_t\}_{k=1}^{\ell},[\![0]\!]_{2t})$.
       - If $\mathsf{reconstruct}(t_1,...,t_n)\neq 0$, the parties call $\mathcal{F}_{\mathsf{checkTranscript}}$ by sending $\{[\![a_k]\!]_t,[\![b_k]\!]_t,[\![c_k]\!]_t\}_{k=1}^{\ell}$ and the transcripts of the protocol to compute each $[\![c_k]\!]_t$ to the functionality, and output whatever they receive back (accept or (i,k)).
       - If $\mathsf{reconstruct}(t_1,\ ...,\ t_n)=\perp$, the parties call $\mathcal{F}_{\mathsf{checkShare}}$ by sending $(t_1,\ ...,\ t_n,\ \{\alpha_k,[\![a_k]\!]_t,[\![b_k]\!]_t,[\![c_k]\!]_t\}_{k=1}^{\ell},\ [\![0]\!]_{2t})$ to the functionality, to receive back a pair (i,k). The parties output (i,k) and halt.

$\underline{\mathsf{search}(u_1,\ldots,u_n,\alpha_1,\ldots,\alpha_\ell,(\llbracket x_1\rrbracket_t,\llbracket y_1\rrbracket_t,\llbracket z_1\rrbracket_t),\ldots,(\llbracket x_\ell\rrbracket_t,\llbracket y_\ell\rrbracket_t,\llbracket z_\ell\rrbracket_t),\llbracket 0\rrbracket_{2t})}$:

1. For $j=1$ to $N-1$:
   (a) The parties run $\mathsf{zeroShare}()$ to receive $\llbracket 0_j\rrbracket_{2t}$.
   (b) The parties locally compute $\llbracket w_j\rrbracket_{2t}=\sum_{k=N(j-1)+1}^{N\cdot j}\alpha_k\cdot(\llbracket z_k\rrbracket_t-\llbracket x_k\rrbracket_t\cdot\llbracket y_k\rrbracket_t)+\llbracket 0_j\rrbracket_{2t}$.
   (c) The parties run $\mathsf{reconstruct}(\llbracket w_j\rrbracket_{2t})$ by broadcasting their shares to each other. Let $w_{j,i}$ be the share broadcasted by P_i.
   (d) If $w_j\neq 0$ or w cannot be reconstructed.
   - If $\frac{\ell}{N}\leq\frac{|C|}{n}$, return $(w_{j,1},\ldots,w_{j,n},\{\alpha_k,\llbracket x_k\rrbracket_t,\llbracket y_k\rrbracket_t,\llbracket z_k\rrbracket_t\}_{k=N(j-1)+1}^{k=N\cdot j},\llbracket 0_j\rrbracket_{2t})$.
   - If $\frac{\ell}{N}>\frac{|C|}{n}$, the parties call $\mathsf{search}(w_{j,1},\ldots,w_{j,n},\{\alpha_k,\llbracket x_k\rrbracket_t,\llbracket y_k\rrbracket_t,\llbracket z_k\rrbracket_t\}_{k=N(j-1)+1}^{k=N\cdot j},\llbracket 0_j\rrbracket_{2t})$ and return the result.
2. (All $N-1$ checks were successful.)
   The parties locally compute $\llbracket 0_N\rrbracket_{2t}=\llbracket 0\rrbracket_{2t}-\sum_{j=1}^{N-1}\llbracket 0_j\rrbracket_{2t}$ and $w_{N,i}=u_i-\sum_{j=1}^{N-1}w_{j,i}$ for each $i\in[n]$.
   - If $\frac{\ell}{N}\leq\frac{|C|}{n}$, return $(w_{N,1},\ldots,w_{N,n},\{\alpha_k,\llbracket x_k\rrbracket_t,\llbracket y_k\rrbracket_t,\llbracket z_k\rrbracket_t\}_{k=\ell-N+1}^{k=\ell},\llbracket 0_N\rrbracket_{2t})$.
   - If $\frac{\ell}{N}>\frac{|C|}{n}$, the parties call $\mathsf{search}(w_{N,1},\ldots,w_{N,n},\{\alpha_k,\llbracket x_k\rrbracket_t,\llbracket y_k\rrbracket_t,\llbracket z_k\rrbracket_t\}_{k=\ell-N+1}^{k=\ell},\llbracket 0_N\rrbracket_{2t})$ and return the result.

Cheating Probability and Security. Assume there is an incorrect triple. The parties will output accept only if in one of the rounds the random linear combination yields 0. Hence, using a similar analysis to Lemma 4.2, we obtain that the successful cheating probability is bounded by $\frac{\log_N n}{\omega_R}$. If the probability is not small enough, then the protocol can be repeated several times.

Our protocol can be proven to securely compute $\mathcal{F}_{\mathsf{vrfy}}$ as stated in the following theorem. The proof is similar to the proof of Theorem 4.3 so we omit the details.

Theorem 5.10. *Protocol 5.9 securely computes $\mathcal{F}_{\mathsf{vrfy}}$ with statistical security in the $\mathcal{F}_{\mathsf{coin}}$-hybrid model against malicious adversaries controlling up to $t<n/3$ parties.*

5.3 Putting It All Together: Cost Analysis

As before, if all triples are correct, and all parties act honestly, then the protocol has a constant number of rounds and $O(n^2)$ communication cost. When cheating took place, the analysis changes. Assume the folding factor is $\sqrt{n}$. Then, the number of rounds until reaching a set of size m/n is 2. The communication cost in these rounds is $O(\sqrt{n}\cdot n^2)$, but can be reduced to $O(n^2)$ (see the optimization at the end of Section 4). Then, the parties call $\mathcal{F}_{\mathsf{checkTranscript}}$ or $\mathcal{F}_{\mathsf{checkShare}}$ a single time. The cost of the protocol to compute each of these functionalities is dominated by n calls to $\mathcal{F}_{\mathsf{Deg2Vrfy}}$ over an input of size $O(\frac{m}{n})$. The communication cost of n calls is $O(n^2\cdot\log(m/n))$ sent elements, the number of rounds is constant, and the amount of work is $n\cdot O(m/n)=O(m)$ arithmetic operations.

Plugging the costs into the Eq. (1), we have that communication cost of the main protocol when using this section's verification protocol is $|C|\cdot|\pi_{mult}|+O(n^3)$ sent elements when all parties act honestly and $|C|\cdot|\pi_{mult}|+O(n^3\cdot\log(|C|/n^2))$ in the worst case. Similarly, by Eq. (2), we have that the number of rounds of the main protocol is $O(\mathsf{depth}(C))+O(n)$. Finally, using Eq. (3) we obtain that the total computational work remains $O(|C|)+O(n\cdot|C|/n)=O(|C|)$ arithmetic operations.

6 Improved Scalability via Gap Amplification

6.1 Gap Amplification

We begin by defining the term *gap* in the context of n-party secure computation with corruption threshold t.

Definition 6.1 (Security Gap)**.** Let $t(n),c(n),g(n)$ be positive functions of n such that $t(n)<n$, and $0<c(n)\leq 1$. We say that t has a c-security gap g if $t(n)<c(n)\cdot n-g(n)$ for any sufficiently large n.

In this section we focus on c-security gaps (gaps for short) with constant $c(n)=1/3$, i.e., the relevance to the threshold $t<n/3$. In Section 6.4 we extend the analysis to wider corruption thresholds such as $t(n)=n/2-o(n)-1$ (an almost honest-majority setting up to sublinear terms). The following lemma gives a bound on the number of eliminations of semi-corrupt pairs required for obtaining a threshold with a certain gap. That is, how many pairs need to be removed so that n' parties remain with at most t' of them corrupt, such that t' has a gap of $g(n')$ from $n'/3$.

Lemma 6.2. *Let n be a number of parties with at most $t(n)=n/3-1$ being corrupted. Let $g(n)<n/3$ be another positive and monotonically increasing function. Then, after removing $3g(n)$ semi-corrupt pairs, the remaining number of corrupted parties t' has a $\frac{1}{3}$-security gap $g(n')$, where n' is the number of remaining parties.*

6.2 Probabilistic Analysis

Our goal is to design a protocol that has the following high-level structure, based on the player elimination framework: (1) For each segment in the circuit, parties securely evaluate the segment and verify the evaluation of the segment. (2) If verification fails, parties identify a semi-corrupt pair, remove it, and go back to Step 1 unless $3g(n)$ pairs of parties have already been eliminated, where n' is the current number of parties. (3) If $3g(n)$ pairs of parties were eliminated, the remaining parties elect a committee of expected size $k(n')$ and go back to Step 1. (4) After all segments are computed, the parties reconstruct the output.

The idea follows from Lemma 6.2: After at most $3g(n)$ pair eliminations, the number of remaining parties n' will have a $g(n')$ gap from $n'/3$ (where $n'=n-6g(n)$). We show that when we reach this gap, electing a random committee of expected size $k(n')$ will have a two-thirds honest majority with overwhelming probability (in the presence of a non-adaptive adversary). If this property does not hold, the security of the protocol is broken, as the adversary holds enough shares to learn the honest parties' private shares. The following lemma proves a constraint on $g(n)$ and $k(n)$ for which we can achieve this goal. We defer its proof to Appendix G.

Lemma 6.3. *Let S be a set of n balls with at most $t=t(n)<n/3-g(n)$ red balls, for some positive, monotonically increasing function $g=g(n)<t(n)$ for all n. Let R denote the set of red balls in S. Let $k(n)<n$ be a positive, monotonically increasing function of n. Let $\mathbf{X}$ be a random subset of S where each item in S is chosen into the set $\mathbf{X}$ with probability k/n. Then:*

1. *The probability $\Pr[||\mathbf{X}|-k(n)|>0.5k]<2^{-\sigma}$ holds for $k>\ln 2\cdot 12\cdot(\sigma+1)$*
2. *For any σ, we have $\Pr[|R\cap\mathbf{X}|\geq|\mathbf{X}|/3]<2^{-\sigma}$ if $g(n)^2\cdot k>2\ln(2)\cdot(\sigma+1)\cdot n^2$*

From Lemma 6.3 we conclude two corollaries:

Corollary 6.4. *When setting $k(n) = n/2$, the committee size satisfies $n/4 \leq |\mathbf{X}| \leq 3n/4$ with probability at least $1-2^{-\sigma}$ when $n \geq \ln 2 \cdot 24 \cdot (\sigma+1)$.*

Corollary 6.5. *Given σ, for any $g(n)$ and $k(n)$ such that $g(n)^2 \cdot k(n) > \ln(2)\sigma n^2$, after eliminating pairs of parties to reach security gap $g(n)$, the probability of electing a robust committee (i.e. with two-thirds honest majority) of size $k(n)$ is $> 1-2^{-\sigma}$.*

Remark 6.6. We can utilize a tradeoff between the guarantee on the committee size and the guarantee on the two-thirds honest majority of the chosen committee. For example, by rejection-sampling the committee until it falls into the range of $n/4 \leq |\mathbf{X}| \leq 3n/4$ such that after w attempts a committee in this range will be sampled with all but $< 2^{-\sigma}$ probability. In this case, the inequality from result (1) of Lemma 6.3 changes to $k > \ln 2 \cdot 12 \cdot (\sigma+1)/w$. Moreover, the inequality from item (2) is now satisfied with all but probability $< 2^{-\sigma} \cdot w$. Alternatively, by setting $\sigma' = \sigma + \log w$ we can use $k > \ln 2 \cdot 12 \cdot (\sigma + \log w + 1)k/w$ and have the inequality from Corollary 6.5 be satisfied with all but probability $< 2^{-\sigma}$ as needed.

Corollary 6.7 (Concrete Parameters)**.** *When taking $k = n/2$, $w = 8$ we get that for $k > \ln 2 \cdot 12 \cdot (\sigma + 1 + \log w)/w = 1.5 \ln 2 \cdot (\sigma+4)$ a committee will be chosen after at most w attempts with size in the range of $[n/4, 3n/4]$ while the committee will be robust when $g(n)^2 \cdot k(n) > 2\ln 2(\sigma+4)n^2$.*

6.3 Application to Secure Computation

In this subsection we present an MPC protocol that leverages Section 6.2 and Corollary 6.5. We describe the protocol and give costs (rounds, communication, computation) in terms of the gap $g(n)$ and sub-committee size $k(n)$. We use statistical security σ and consider $t = n/3 - 1$; hence Lemma 6.3 implies $g(n)^2 \cdot k(n) > \ln(2)\sigma n^2$. The formal protocol description appears in Protocol 6.8.

Protocol 6.8. Secure Computation with $t = n/3 - 1$ and Committee Election

- **Public parameters:** Committee size function $k(n)$ and gap function $g(n)$. Ring R, circuit C of size $|C|$ over R. Sampling rejection repetition threshold w. Let π_{mult} be a multiplication protocol that takes two shared inputs and outputs a shared output.
- **Input**: Each P_i holds a private input x_k for each input wire w_k associated with P_i.

1. **Input Phase:** For each input wire w_k provided by party P_i, party P_i secret-shares x_k with threshold t across the parties.
2. The parties split the circuit C into $3g(n)$ segments $S_1, \ldots, S_{3g(n)}$. Each segment S_i is of size $|C|/(3g(n))$.
3. **Computation Phase:** For each segment S_i or until $3g(n)$ pairs are eliminated:
   (a) The parties evaluate the segment S_i using π_{mult}.
   (b) The parties call $\mathcal{F}_{\mathsf{vrfy}}$ (Functionality 3.1) on their shares of multiplication gates in S_i as well as their view from the execution of π_{mult}.
   (c) If the parties receive accept from $\mathcal{F}_{\mathsf{vrfy}}$, they continue to the next segment.
   (d) Otherwise, they receive a pair of parties (P_i, P_j). Then, the parties P_i, P_j are eliminated and the inputs to the segment are reshared to threshold $t-1$ across the remaining $n-2$ parties using the $\mathsf{refresh}$ procedure.

4. If all segments were computed successfully, the parties proceed to Step 8.
5. **Committee Election Phase:** If $3g(n)$ pairs are eliminated, the remaining parties call $\mathcal{F}_{\mathsf{coin}}$ to elect a committee of expected size $k(n)$, with each party chosen into the committee with probability $k(n)/n$. If the chosen committee is of size $> 1.5 \cdot k(n)$ or $< 0.5 \cdot k(n)$, re-elect the committee. If more than w re-elections took place, abort. Let $s(n) \in [0.5k(n), 1.5k(n)]$ be the size of the chosen committee. Let $P'_1,\ldots,P'_{s(n)}$ be the elected committee.
6. The parties run the $\mathsf{refresh}$ protocol to reshare all inputs to the current segment with threshold $s(n)/3$ across $P'_1,\ldots,P'_{s(n)}$.
7. The committee $P'_1,\ldots,P'_{s(n)}$ evaluates the remaining segments one by one as instructed in the computation phase.
8. **Output Phase:** For each output w_i intended to party P_i, the parties in the committee run $\mathsf{reconstruct}(\llbracket w_i \rrbracket_{s(n)/3}, i)$ (note that the reconstruction always succeeds). Then, P_i outputs w_i.

6.3.1 Cost Analysis

The protocol requires executing $\mathsf{depth}(C)$ rounds for the semi-honest protocol to evaluate a circuit C. At the end of each segment, a $O(1)$ round verification protocol is executed, with at most $3g(n)$ eliminations of pairs of parties. Then, the remaining committee continues the computation which incurs additional $O(k(n))$ rounds of verification. Overall, we get $O(\mathsf{depth}(C) \cdot \mathsf{Round}(\pi_{\mathsf{mult}}) + k(n) + g(n))$ rounds. In terms of communication, the semi-honest protocol incurs $O(|C| \cdot \mathsf{Comm}(\pi_{mult}))$ communication. The verification protocol is run $3g(n)$ times by the n parties and then an additional $k(n)/3$ times by the committee, since the committee has at most $k(n)/3$ corrupted parties. This results, overall, in communication of $O\big(g(n) \cdot n^2 \cdot \log(|C|/(n \cdot g(n)))\big)$, plus $O\big(k(n)^3 \cdot \log(|C|/(n \cdot g(n)))\big)$ communication across all parties together. In terms of computation, we retain the scalability property with $O(|C|)$ additional computation per party: each party participates in at most $3g(n)$ eliminations with $O(n)$ zk-FLIOP verifications of sub-segments of size $|C|/(n \cdot g(n))$, and then in at most $k(n)/3$ eliminations with $O(k(n))$ zk-FLIOP verifications of sub-segments of size $|C|/k(n)^2$.

Concrete Choice of $g(n)$ and $k(n)$: Balancing parameters yields $g(n) = k(n) = \sqrt[3]{\ln(2)\sigma n^2}$. Hence the round complexity is $O(\mathsf{depth}\,(C) \cdot \mathsf{Round}(\pi_{\mathsf{mult}}) + n^{2/3})$, and total communication is $O\big(|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) + n^{8/3} \log(|C|/n^{5/3})\big)$. Per-party computation remains $O(|C| \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}))$.

Recursive Committee Selection: We reduce round and communication costs even further by employing the committee selection process recursively. The protocol begins as usual with $n_0 = n$ parties by eliminating $O(g(n))$ pairs of parties to achieve the $g(n)$ gap. Then, the remaining parties elect a committee of expected size $k(n)$. This committee is guaranteed with overwhelming probability (in σ) to have a two-thirds honest majority and to be smaller than $1.5k(n)$. Let n_1 be the size of the elected committee. At this point, we can run the protocol *recursively* with n_1 parties by executing the protocol until a gap of at most $g(n_1)$ is achieved after eliminations of $O(g(n_1))$ pairs of parties, and so on. We stress that the size of segments does not change after recursive calls as it may increase the overall communication complexity. The cost analysis is now as follows, assuming the circuit is regular (i.e. as if all segments are of equal size, width, and depth). In the analysis, we denote by ℓ the number of recursive committee elections and by $L = \sum_{i=0}^{\ell-1} g\big(k^i(n)\big)$ the total number of segments computed in the circuit. For simplicity we also assume that the size of the

elected committee is always the maximal size in the allowed range, that is $1.5k = 3n/4$. Whenever it is smaller, fewer resources will be consumed. In fact, for complexity analysis, one can just think of $k(n) = 3n/4$. We summarize the costs of the recursive protocol in the following theorem and defer proof to Appendix G.

Remark 6.9. We stress that in the recursive case parameter choices are slightly different to bound the possible error probability across all rounds. Therefore, we need each round to introduce error of $< 2^{-\sigma-\log r}$ when r is the number of recursive calls. Since the number of recursive calls is with overwhelming probability $< \log n$ we get that from Lemma 6.3 we need to set $g(n)^2 \cdot k(n) > \ln(2)(\sigma+1+\log\log n)n^2$, that is: when using $k(n) = n/2$ we now eliminate parties to create a gap of $g(n) = 2\sqrt{\ln 2(\sigma+1+\log\log n)n/3}$.

Theorem 6.10. *Let C be a circuit of size $|C|$ and depth $\mathsf{depth}(C)$. Let π_{mult} be a secure multiplication protocol with round complexity $\mathsf{Round}(\pi_{\mathsf{mult}})$, communication complexity $\mathsf{Comm}(\pi_{\mathsf{mult}})$ and computational complexity $\mathsf{Comp}(\pi_{\mathsf{mult}})$. Let $g(n)$ and $k(n)$ be positive monotonically increasing functions such that $g(n)^2 \cdot k(n) > \ln(2)(\sigma+1+\log\log n)n^2$. Then, Protocol 6.8 securely computes C with statistical security σ against a static adversary corrupting at most $t = n/3-1$ parties, with:*

- *Round complexity of $O(L \cdot (\mathsf{depth}(C) \cdot \mathsf{Round}(\pi_{\mathsf{mult}})/g(n)) + L)$*
- *Communication complexity across all parties together of $O\left(|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) \sum_{i=0}^{\ell-1} \left(\frac{g(k^i(n))}{g(n)}\right) + L \cdot n^2 \log(|C|/(n \cdot g(n)))\right)$*
- *Per party computation of $O(L \cdot (|C|/g(n)) \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}) + L \cdot (|C|/g(n)))$*

where ℓ is the number of recursive committee elections and $L = \sum_{i=0}^{\ell-1} g\left(k^i(n)\right)$ is the total number of segments computed in the circuit.

Concrete Parameter Choices: For $k(n) = n/2$ and $g(n) = \sqrt{2\ln(2)\sigma n \log\log n}$, we get optimal costs: $O(\mathsf{depth}(C) \cdot \mathsf{Round}(\pi_{\mathsf{mult}}) + \sqrt{n})$ rounds with recursion depth $< \log n$, total communication complexity of $O\left(|C| \cdot \mathsf{Comm}(\pi_{\mathsf{mult}}) + \sqrt{n^5} \log\left(|C|/\sqrt{n^3}\right)\right)$ across all parties, and computational complexity $O(|C| \cdot \mathsf{Comp}(\pi_{\mathsf{mult}}) + |C|)$ per party.

Practicality of Our Approach. Is gap amplification practical, and for which n? Our optimal choice $g(n) = \sqrt{4\ln(2)(\sigma+1)n}$ must satisfy $g(n) < t(n)$ (Lemma 6.3), so $\sqrt{4\ln(2)(\sigma+1)n} < n/3$, which simplifies to $n > 36\ln(2)(\sigma+1)$. For the common choice $\sigma = 40$ this gives roughly $n \geq 1000$. More generally, taking $k(n) = n/c$ and $g(n) = \sqrt{c \cdot \ln(2)\sigma n}$ yields $n \geq 498c$. We expect further optimizations could lower these thresholds.

6.4 Relevance to Honest Majority

We briefly discuss how gap amplification applies to the (almost) honest majority setting where $t < n(1/2 - h(n)) - 1$ for $h(n) = o(1)$. We defer proofs to Appendix G. Our main observation is that the technique from Lemma 6.2 can achieve a gap of $g(n)$ from $n(1/2-h(n))$ (rather than from $n/3$), and this requires eliminating at most $\frac{g(n)}{2h(n)}$ semi-corrupt pairs.

Lemma 6.11. *Let n be a number of parties and $g(n), h(n)$ be positive functions of n such that $h(n) = o(1)$ is monotonically decreasing and $g(n)$ is monotonically increasing. Let the number of corrupted parties be $t(n) = \left(\frac{1}{2} - h(n)\right) \cdot n - 1$. Then, by eliminating at most $\frac{g(n)}{2h(n)}$ pairs of semi-corrupt parties, the number of remaining corrupt parties t' has a $\left(\frac{1}{2} - h(n)\right)$-security gap of $g(n)$.*

Next, by adapting the analysis of Lemma 6.3 to threshold $t(n)$, we get that after achieving gap $g(n)$, electing a committee of size $k(n)$ yields (with overwhelming probability in σ) at most $t(k(n))$ corrupted parties in the committee, thus preserving the threshold.

Lemma 6.12. *Let S be a set of n balls with at most $t = t(n) < n(\frac{1}{2} - h(n)) - g(n)$ red balls, for some positive, monotonically increasing functions $g = g(n) < t(n)$ and $h = h(n) = o(1)$ for all n. Let R denote the set of red balls in S. Let $k = k(n) < n$ be a positive, monotonically increasing function of n. Let $\mathbf{X}$ be a random subset of S where each item in S is chosen into the set $\mathbf{X}$ with probability k/n. Then:*

1. *The probability $\Pr[||\mathbf{X}| - k(n)| > 0.5k] < 2^{-\sigma}$ holds for $k > \ln 2 \cdot 12 \cdot (\sigma + 1)$*
2. *For any σ, $\Pr\left[|R \cap \mathbf{X}| \geq |\mathbf{X}| \cdot \left(\frac{1}{2} - h(n)\right)\right] < 2^{-\sigma}$ if $g(n)^2 \cdot k > 2\ln(2) \cdot (\sigma + 1) \cdot n^2$*

Thus, we can take $g(n) = \sqrt{2\ln 2\sigma \cdot n \log\log n}$ and $k(n) = n/2$ to achieve similar benefits as in the two-thirds honest majority setting. Since this gap amplification technique is compatible with any protocol following the player elimination framework, we get:

Corollary 6.13. *By applying any protocol compatible with the player elimination framework with corruption threshold $t < n(1/2 - h(n)) - 1$ for $h(n) = o(1)$ and applying the gap amplification technique from Lemma 6.11 and Lemma 6.12 we can achieve additional round complexity of $O(\sqrt{n} \cdot h(n)^{-1})$. For any $h(n) = \omega(n^{-1/2})$ this results in $o(n)$ additional rounds, at the cost of non-adaptive security.*

References

[AAPP23] Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Detect, pack and batch: Perfectly-secure MPC with linear communication and constant expected time. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 251–281, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.

[ADEN21] Mark Abspoel, Anders P. K. Dalskov, Daniel Escudero, and Ariel Nof. An efficient passive-to-active compiler for honest-majority MPC over rings. In Kazue Sako and Nils Ole Tippenhauer, editors, *ACNS 21International Conference on Applied Cryptography and Network Security, Part II*, volume 12727 of *LNCS*, pages 122–152, Kamakura, Japan, June 21–24, 2021. Springer, Cham, Switzerland.

[BBC+19] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear PCPs. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 67–97, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.

[BFO12] Eli Ben-Sasson, Serge Fehr, and Rafail Ostrovsky. Near-linear unconditionally-secure multiparty computation with a dishonest minority. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 663–680, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Berlin, Heidelberg, Germany.

[BGIN19] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Practical fully secure three-party computation via sublinear distributed zero-knowledge proofs. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 869–886, London, UK, November 11–15, 2019. ACM Press.

[BGIN20] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Efficient fully secure computation via distributed zero-knowledge proofs. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 244–276, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.

[BGIN21] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Sublinear GMW-style compiler for MPC with preprocessing. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 457–485, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

[BGIN22] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Secure multiparty computation with sublinear preprocessing. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 427–457, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

[BGP92] Piotr Berman, Juan A. Garay, and Kenneth J. Perry. Bit optimal distributed consensus. In *Computer science*, pages 313–321. Birkhäuser, 1992.

[BGT13] Elette Boyle, Shafi Goldwasser, and Stefano Tessaro. Communication locality in secure multi-party computation - how to run sublinear algorithms in a distributed setting. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 356–376, Tokyo, Japan, March 3–6, 2013. Springer, Berlin, Heidelberg, Germany.

[BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10, Chicago, IL, USA, May 2–4, 1988. ACM Press.

[BTH08] Zuzana Beerliová-Trubíniová and Martin Hirt. Perfectly-secure MPC with linear communication complexity. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 213–230, San Francisco, CA, USA, March 19–21, 2008. Springer, Berlin, Heidelberg, Germany.

[Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.

[CCD88] David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In *20th ACM STOC*, pages 11–19, Chicago, IL, USA, May 2–4, 1988. ACM Press.

[CGH+18] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 34–64, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

[Cle86] Richard Cleve. Limits on the security of coin flips when half the processors are faulty (extended abstract). In *18th ACM STOC*, pages 364–369, Berkeley, CA, USA, May 28–30, 1986. ACM Press.

[CW89] Brian A. Coan and Jennifer L. Welch. Modular construction of nearly optimal byzantine agreement protocols. In Piotr Rudnicki, editor, *8th ACM PODC*, pages 295–305, Edmonton, Alberta, Canada, August 14–16, 1989. ACM.

[DEK21] Anders P. K. Dalskov, Daniel Escudero, and Marcel Keller. Fantastic four: Honest-majority four-party secure computation with malicious security. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 2183–2200. USENIX Association, August 11–13, 2021.

[DEN22] Anders P. K. Dalskov, Daniel Escudero, and Ariel Nof. Fast fully secure multi-party computation over any ring with two-thirds honest majority. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 653–666, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.

[DEN24] Anders P. K. Dalskov, Daniel Escudero, and Ariel Nof. Fully secure MPC and zk-FLIOP over rings: New constructions, improvements and extensions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 136–169, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.

[DKM+17] Varsha Dani, Valerie King, Mahnush Movahedi, Jared Saia, and Mahdi Zamani. Secure multi-party computation in large networks. *Distrib. Comput.*, 30(3):193–229, 2017.

[DKMS12] Varsha Dani, Valerie King, Mahnush Movahedi, and Jared Saia. Brief announcement: breaking the $O(nm)$ bit barrier, secure multiparty computation with a static adversary. In Darek Kowalski and Alessandro Panconesi, editors, *31st ACM PODC*, pages 227–228, Funchal, Madeira, Portugal, July 16–18, 2012. ACM.

[DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 572–590, Santa Barbara, CA, USA, August 19–23, 2007. Springer, Berlin, Heidelberg, Germany.

[DOS18] Ivan Damgård, Claudio Orlandi, and Mark Simkin. Yet another compiler for active security or: Efficient MPC over arbitrary rings. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 799–829, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

[FL19] Jun Furukawa and Yehuda Lindell. Two-thirds honest-majority MPC for malicious adversaries at almost the cost of semi-honest. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1557–1571, London, UK, November 11–15, 2019. ACM Press.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press.

[GIP+14] Daniel Genkin, Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In David B. Shmoys, editor, *46th ACM STOC*, pages 495–504, New York, NY, USA, May 31 – June 3, 2014. ACM Press.

[GIP15] Daniel Genkin, Yuval Ishai, and Antigoni Polychroniadou. Efficient multi-party computation: From passive to active security via secure SIMD circuits. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 721–741, Santa Barbara, CA, USA, August 16–20, 2015. Springer, Berlin, Heidelberg, Germany.

[GLO+21] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. ATLAS: Efficient and scalable MPC in the honest majority setting. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 244–274, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

[GLS19] Vipul Goyal, Yanyi Liu, and Yifan Song. Communication-efficient unconditional MPC with guaranteed output delivery. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 85–114, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.

[GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229, New York City, NY, USA, May 25–27, 1987. ACM Press.

[Gol04] Oded Goldreich. *Foundations of Cryptography: Basic Applications*, volume 2. Cambridge University Press, Cambridge, UK, 2004.

[GSZ20] Vipul Goyal, Yifan Song, and Chenzhi Zhu. Guaranteed output delivery comes free in honest majority MPC. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 618–646, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.

[HMP00] Martin Hirt, Ueli M. Maurer, and Bartosz Przydatek. Efficient secure multi-party computation. In Tatsuaki Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 143–161, Kyoto, Japan, December 3–7, 2000. Springer, Berlin, Heidelberg, Germany.

[IKP+16] Yuval Ishai, Eyal Kushilevitz, Manoj Prabhakaran, Amit Sahai, and Ching-Hua Yu. Secure protocol transformations. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 430–458, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Berlin, Heidelberg, Germany.

[IPS08] Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. Founding cryptography on oblivious transfer - efficiently. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 572–591, Santa Barbara, CA, USA, August 17–21, 2008. Springer, Berlin, Heidelberg, Germany.

[ISN89] Mitsuru Ito, Akira Saito, and Takao Nishizeki. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)*, 72(9):56–64, 1989.

[KPRS22] Nishat Koti, Arpita Patra, Rahul Rachuri, and Ajith Suresh. Tetrad: Actively secure 4pc for secure training and inference. In *NDSS*, 2022.

[LDH+23] Yun Li, Yufei Duan, Zhicong Huang, Cheng Hong, Chao Zhang, and Yifan Song. Efficient 3PC for binary circuits with application to maliciously-secure DNN inference. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 5377–5394, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.

[LN17] Yehuda Lindell and Ariel Nof. A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 259–276, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.

[Sha79] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press.

A Check Consistency of Shares

The protocol is described in Protocol A.1.

Protocol A.1. ss.check() **- Check Consistency of sharings**

- **Input**: $[\![v_1]\!]_d,\ldots,[\![v_m]\!]_d$ and an index i of the dealer party.
- **The protocol**:
  1. P_i chooses a random v_0 and runs $\mathsf{share}(v_0)$ with threshold d towards all parties.
  2. The parties call $\mathcal{F}_{\mathsf{coin}}$ to receive random $\alpha_1,\ldots,\alpha_m$.
  3. The parties locally compute $[\![u]\!]_d = [\![v_0]\!]_d + \sum_{k=1}^{m} \alpha_k \cdot [\![v_k]\!]_d$.
  4. The parties run $\mathsf{reconstruct}([\![u]\!]_d)$ by broadcasting (via $\mathcal{F}_{\mathsf{bc}}$) their shares.
  5. If u can be reconstructed, the parties output accept. Otherwise, P_i broadcasts (via $\mathcal{F}_{\mathsf{bc}}$) an index k of a party which published an incorrect share, and the parties output (i,k).

  (Note that steps 2-4 can be repeated multiple times to reduce the cheating probability. In this case, the parties output accept only if all instances end with a success reconstruction.)

B Main Protocol and Proof of Security

Input Sharing Step. At the end of this step, the parties will hold a consistent d-out-of-n secret sharing of each input.

1. The parties set $n' = n$ and $d = t$, where $n = 3t+1$.
2. For the jth input x_j held by P_i: party P_i runs $\mathsf{vss.share}(x_j)$ with threshold d. If the protocol outputs a semi-corrupt pair (i,k), then P_i broadcasts (via $\mathcal{F}_{\mathsf{bc}}$) the share intended to P_k.

Computing the Next Segment. This step begins with the parties holding a consistent secret sharing of the values on the input wires of the segment.

3. The parties compute the segment gate after gate in some predetermined topological order. Linear gates are computed locally and multiplication gates are computed using a protocol π_{mult} that realizes the ideal functionality $\mathcal{F}_{\mathsf{Mul}}^{+}$ (i.e., secure up to an additive attack).
4. The parties send their inputs and outputs of all multiplication gates to $\mathcal{F}_{\mathsf{vrfy}}$. If $\mathcal{F}_{\mathsf{vrfy}}$ sent accept, then they proceed to the next segment. Otherwise, they receive from $\mathcal{F}_{\mathsf{vrfy}}$ a pair of parties (P_i,P_j) to eliminate. The parties then run the elimination and recovery subprotocol $\mathsf{refresh}(i,j)$, set $n' = n'-2$, $d = d-1$ and go back to Step 3.

Output Reconstruction. At the beginning of this step, the parties hold a d-out-of-n' secret sharing of each output. Then, for each output o_k intended to party P_i, the parties run $\mathsf{reconstruct}([\![o_k]\!]_d, i)$ (note that since $d < n/3$, reconstruction will succeed).

Theorem B.1. *Let R be a finite ring, let f be an n-party functionality represented by an arithmetic circuit over R, and let $t \in \mathbb{N}$ be a security threshold parameter such that $n = 3t+1$. Then, our main*

protocol, as described in the text, securely computes f in the ($\mathcal{F}_{\mathsf{vrfy}}$)-hybrid mode with statistical security, in the presence of malicious adversaries controlling up to t parties.

Proof. Let $\mathcal{S}$ be the ideal world simulator and let $\mathcal{A}$ be the real-world adversary. In the simulation, $\mathcal{S}$ plays the role of the honest parties, and the ideal functionality $\mathcal{F}_{\mathsf{vrfy}}$. The simulation in each of the steps works as follows:

- Input sharing step: In this step, $\mathcal{S}$ sets the input of each honest party to be 0. For inputs held by corrupted parties, $\mathcal{S}$ receives from $\mathcal{A}$ shares intended to the honest parties. Then, it follows the instructions of the protocol. Observe that since an honest majority exists, $\mathcal{S}$ receives enough shares that allow it to compute the secrets and extract the corrupted parties' inputs. In addition, it knows all the shares held by corrupted parties on each of the input wires.

- Segment computation: In this step, $\mathcal{S}$ follows the instructions of the simulator that exists for the multiplication protocol. Since $\mathcal{S}$ knows the inputs' shares to each gate, it knows whether cheating took place. Moreover, since π_{mult} is secure up to an additive attack, it can extract the additive error on the output wires of each multiplication gate and the corrupted parties' shares on these wires. For addition gates, it simply adds the corrupted parties' shares

  Finally, $\mathcal{S}$ plays the role of $\mathcal{F}_{\mathsf{vrfy}}$. If no cheating took place (i.e. all additive errors are 0), then it sends accept to $\mathcal{A}$, and otherwise, it waits to receive a semi-corrupt pair to be removed from $\mathcal{A}$. In the former case, it waits for $\mathcal{A}$ to send back accept which means that the execution proceeds to the next computation, or to receive from $\mathcal{S}$ a semi-corrupt pair to remove.

  In the case that a semi-corrupt pair was located and the segment is recomputed, the elimination and recovery subprotocol is executed. In the simulation, $\mathcal{S}$ simply follows the protocol instructions while playing the role of the honest parties.

- Output reconstruction: Note that throughout the simulation, $\mathcal{S}$ knows the corrupted parties' shares on all wires. When reaching the output wires, the simulator $\mathcal{S}$ sends the corrupted parties' inputs to the trusted party computing f, to receive back their outputs. Then, $\mathcal{S}$ replaces the shares held by honest parties with new random shares that, together with the corrupted parties' shares (known to $\mathcal{S}$), would reconstruct to the output received from the trusted party. Then, $\mathcal{S}$ plays the role of the honest parties sending $\mathcal{A}$ their shares. Finally, $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs.

Observe that the only difference between the simulated and the real execution is the honest parties' inputs. However, by the secrecy of the secret sharing scheme, this makes no difference in the input sharing step. In the circuit's computation step, by the security properties of π_{mult}, the view of the corrupted parties is the same regardless of the used input. Thus, the view of $\mathcal{A}$ up to the last step is distributed the same in both executions. Finally, in the output reconstruction step, by the secrecy of the secret sharing scheme and since $\mathcal{A}$'s view till this step is the same in both executions, it follows that the honest parties' shares of the outputs are identically distributed in both executions. This concludes the proof. □

C A Protocol for the refresh Procedure

The way to realize this procedure depends on the secret sharing scheme and semi-honest protocol. Ishai et al. [IKP+16] proposed a generic method for refreshing with cost that grows linearly with

the width of the circuit. Boyle et al. [BGIN20] showed how the removal can be done essentially for free, when working with replicated secret sharing. Both the above methods follow the blueprint of resharing the secrets on the output layer of the previous segment with a lower threshold. Goyal et al. [GSZ20] presented a different approach where the threshold remains the same, but the shares of eliminated parties are set to be 0, using a refresh protocol, carried-out for each multiplication. Our compiler can be combined with any of these methods to obtain an end-to-end fully secure protocol. Nevertheless, the above methods are all for the setting of $t<n/2$. We now describe an optimized method for refreshing that leverages the $t<n/3$ setting, with communication cost that scales linearly with the number of inputs to the layer.

Recall that the goal of this protocol is to convert $[\![x]\!]_d$ to $[\![x]\!]_{d'}$ where $d'<d$. Moreover, $[\![x]\!]_d$ is shared across a set S of n parties, while $[\![x]\!]_{d'}$ should be shared across a set S' of size $n'<n$ parties (but in both sets, two-thirds of the parties are honest).

The idea behind the protocol is as follows. The parties in S' prepare a pair of random sharings of the form $[\![r]\!]_d$ and $[\![r]\!]_{d'}$, where the former is shared across S and the latter is shared across parties in S' only. Then, the parties run $\mathsf{reconstruct}([\![x]\!]_d-[\![r]\!]_d)$ to obtain $x-r$ and then locally compute $[\![x]\!]_{d'}=[\![r]\!]_{d'}+x-r$.

In the actual protocol, the parties wish to convert a batch of sharings $[\![x_1]\!]_d,\ldots,[\![x_\ell]\!]_d$ to $[\![x_1]\!]_{d'},\ldots,[\![x_\ell]\!]_{d'}$ (these correspond to values on the input wires to a circuit's segment). The parties can prepare the pairs $([\![r_1]\!]_d,[\![r_1]\!]_{d'}),\ldots,([\![r_\ell]\!]_d,[\![r_\ell]\!]_{d'})$ with linear communication complexity using Hyper-invertible matrices [BTH08] and also use the batch reconstruction technique from [BTH08] to reconstruct $x_1-r_1,\ldots,x_\ell-r_\ell$ with linear communication complexity. Moreover, since $d<n/3$, the reconstruction cannot fail. Finally, the parties can determine whether a party has cheated when preparing the random sharings using $\mathsf{ss.check}()$. Here again we can use the fact that $d,d'<n/3$ to identify whether the dealer has cheated. Since all dealers are in S', we are guaranteed that in case of cheating a new corrupted party will be identified (recall that in our protocol S' includes the parties that remain active after parties were eliminated).

D Proof of Theorem 4.3

Proof. Let $\mathcal{S}$ be the ideal-world simulator and let $\mathcal{A}$ be the real-world adversary. The simulation begins with $\mathcal{S}$ receiving $d_k=z_k-x_k\cdot y_k$ for all $k\in[m]$ and the corrupted parties' shares of x_k,y_k and z_k from the trusted party computing $\mathcal{F}_{\mathsf{vrfy}}$. In the simulation, $\mathcal{S}$ plays the role of $\mathcal{F}_{\mathsf{coin}}$ choosing and handing $\mathcal{A}$ the random coefficients $\alpha_1,\ldots,\alpha_m$. Then, it works according to the following guidelines:

- *Simulating* $\mathsf{reconstruct}()$ *executions*: Note that knowing d_k and α_k for each k, $\mathcal{S}$ can compute the secrets to be reconstructed as well as the corrupted parties' shares. Hence, it remains to choose shares for the honest parties. $\mathcal{S}$ thus chooses random shares that, together with corrupted parties' shares will reconstruct to the correct value. Note that since all shares are randomized by adding $[\![0]\!]_{2t}$, the view of the adversary is identical to its view in the real execution. If the honest parties output accept in the simulation, although there exists k such that $d_k\neq 0$, the simulator $\mathcal{S}$ outputs fail and halts.

- *Simulating* $\mathsf{zeroShare}()$: recall that in this subprotocol, each party shares 0 with threshold $2t$, and then the parties check for consistency. If the check fails, the parties should output a semi-corrupt pair. Hence, for sharings dealt by honest parties, $\mathcal{S}$ simply chooses random shares for the corrupted parties and sends them to $\mathcal{A}$. Then, in the consistency check, it simulates reconstruction

by choosing random shares for the honest parties, that together with the corrupted parties' shares will reconstruct to 0. If a corrupted party P_k sent an incorrect share in the reconstruction, $\mathcal{S}$ simulates the honest dealer broadcasting k to the parties.

For sharings dealt by corrupted parties, it receives from $\mathcal{A}$ the honest parties' shares. Then, it follows the protocol instructions. If the dealer sends inconsistent shares or shares any value that is not 0, the honest parties detect it, except for a small probability. If cheating has been detected, $\mathcal{S}$ receives from $\mathcal{A}$ an index k of some party.

If the result of the simulated execution is a pair (i,k) (where i is the index of the dealer), then $\mathcal{S}$ sends (i,k) to the trusted party computing $\mathcal{F}_{\mathsf{vrfy}}$ and halts.

- *Simulating* $\mathcal{F}_{\mathsf{findPair}}$: Here $\mathcal{S}$ plays the role of the ideal functionality and needs to generate the correct output.

  When $\mathcal{F}^2_{\mathsf{findPair}}$ is called, $\mathcal{S}$ can compute the correct share t_i that should be sent by each corrupted party P_i, since $t_i = c_i - a_i \cdot b_i + 0_i$ (where c_i, a_i, b_i and 0_i are shares held by P_i) and so can identify the cheater. Then, it sends the index of the cheater to $\mathcal{A}$ to receive back an index of another party. Finally, it hands this pair to the trusted party computing $\mathcal{F}_{\mathsf{vrfy}}$ and halts.

  When $\mathcal{F}^1_{\mathsf{findPair}}$ is called, $\mathcal{S}$ simulates the ideal functionality handing the exchanged messages between honest and corrupted parties to $\mathcal{A}$ (note that these are given to $\mathcal{S}$ by $\mathcal{F}_{\mathsf{vrfy}}$ for the lowest index k for which $d_k \neq 0$). Upon receiving back a semi-corrupt pair from $\mathcal{A}$, it hands it to $\mathcal{F}_{\mathsf{vrfy}}$ and halts.

The only difference between the simulated and a real execution is the event where $\mathcal{S}$ outputs fail. Note that this happens when $\sum_{k=1}^{m} \alpha_k \cdot (z_k - x_k \cdot y_k) = 0$ even though there exists k for which $d_k = z_k - x_k \cdot y_k \neq 0$ or if the last triple to verify is found to be correct. However, by Lemma 4.2, this happens with a constant probability. Hence, the protocol achieves statistical security as claimed by the theorem. This concludes the proof. □

E Resolve Inconsistency of a Secret Sharing

The protocol is formally described in Protocol E.1.

Protocol E.1. $(j,k) \leftarrow \mathsf{ss.check}([\![x]\!]_d)$**: Resolving Inconsistency**

Input: The parties hold a sharing $[\![x]\!]_d$ which was found to be inconsistent. Let x_i be the share of x that was published by P_i. Let $[\![x^i]\!]_d$ be a consistent sharing that was shared by P_i such that $[\![x]\!]_d = \sum_{i=1}^{n} [\![x^i]\!]_d$.

The protocol:

1. Each party P_i chooses a random r^i and runs $\mathsf{share}(r^i)$ with threshold d to the other parties.
2. The parties locally compute $[\![r]\!]_d = \sum_{i=1}^{n} [\![r^i]\!]_d$ and run $\mathsf{reconstruct}([\![r]\!]_d)$ by broadcasting their shares. Let r_i be the share published by P_i.
3. If r cannot be reconstructed, due to inconsistency, then the parties set $[\![z]\!]_d = [\![r]\!]_d$ and $[\![z^i]\!]_d = [\![r^i]\!]_d$ for each $i \in [n]$.
   Otherwise, they set $[\![z]\!]_d = [\![x]\!]_d + [\![r]\!]_d$ and $[\![z^i]\!]_d = [\![x^i]\!]_d + [\![r^i]\!]_d$ for each $i \in [n]$.

4. For each $i \in [n]$, the parties run $\mathsf{reconstruct}(\llbracket z^i \rrbracket_d)$ by broadcasting their shares. Let z^i_j be the share of z^i published by P_j. Then:
   - If there exists $i \in [n]$ for which $\mathsf{reconstruct}(\llbracket z^i \rrbracket_d) = \perp$, then P_i broadcasts (accuse,k) where k is an index of a party P_k that published an incorrect share. In this case, the parties output (i,k).
   - Otherwise, the parties proceed to the next step.
5. Let j be the minimal index for which $\sum_{i=1}^n z^i_j \neq z_j$. Then, the parties output (j,k) where k is the smallest index such that $k \neq j$.

Correctness. To see why the protocol is correct, consider the following cases:

- *Case 1: r could not be reconstructed.* In this case, $z = r = \sum_{i=1}^n r^i$ and the parties reconstruct r^i for each $i \in [n]$. If the reconstruction of any of these values fails, the party who dealt that value can identify the cheater. If the dealer itself is corrupted, then regardless of the other party, the output is semi-corrupt. If all r^is were successfully reconstructed, then it means that parties can compute each party P_j's share r_j of r by taking $\sum_{i=1}^n r^i_j$. However, since r could not be reconstructed, then it means that there must be some party P_j for which $r_j \neq \sum_{i=1}^n r^i_j$. Hence, the parties can locate a corrupted party P_j. In this case, any pair (j,k) is a semi-corrupt pair.
- *Case 2: r was successfully reconstructed.* In this case, $z = x + r = \sum_{i=1}^n (x^i + r^i)$. Note that since the parties have already published their shares of x and r, then all shares z_i are known. Then, the parties also reconstruct z^i for each $i \in [n]$. If for some i, the reconstruction fails, then as in the previous case, the dealer of that value can identify the cheater. Otherwise, all shares are consistent, which means that all shares $\sum_{i=1}^n z^i_j$ form a consistent sharing of z. However, we know that the shares of z are inconsistent. Hence, there must be some j for which $z_j \neq \sum_{i=1}^n z^i_j$, implying that P_j is corrupted. Then, we can simply add any other party to obtain a semi-corrupt pair as required.

Privacy: Simulating the Protocol. We describe a simulator $\mathcal{S}$ that receives as input all shares of x, all shares of x_i that were shared by a corrupted P_i, and all shares held by corrupted parties of x_j that were shared by an honest P_j. In the simulation, $\mathcal{S}$ sends the adversary $\mathcal{A}$, controlling the corrupted parties, random shares as their shares of r^j for each honest party P_j. In addition, it receives the honest parties shares of r^i for each corrupted party P_i. This suffices for $\mathcal{S}$ to determine whether r is consistent or not. Then:

- *Case 1: r is not consistent.* In this case, $\mathcal{S}$ chooses a random r^j for each honest P_j and chooses shares of each r^j for the honest parties, given the corrupted parties' shares dealt above. Then, it simulates the protocol by playing the role of the honest parties. Note that since in this case, the protocol is executed over random independent shares, the simulation is perfect.
- *Case 2: r is consistent.* In this case, $\mathcal{S}$ can compute the corrupted parties' shares of r^i for each corrupted P_i. This means that it knows the corrupted parties' shares of all r^i and can compute their shares of r. Then, $\mathcal{S}$ chooses a random r, and chooses random shares to the honest parties that will reconstruct, together with the corrupted parties' shares, to r.

  At this point, $\mathcal{S}$ knows $z = x + r$, all shares of z and the shares held by corrupted parties of each $z^i = x^i + r^i$. Then, $\mathcal{S}$ chooses random z^i under the constraint that $z = \sum_{i=1}^n z^i$ and chooses the

honest parties shares that together with the corrupted parties shares will reconstruct to each z^i. From this point on, $\mathcal{S}$ can simulate the protocol by playing the role of the honest parties running the protocol's instructions.

It is easy to see that $\mathcal{A}$'s view in the simulation is distributed the same as in the real execution, due to the masking with a random uniformly chosen r^i.

F Converting Sharings to a Different Threshold

The formal description appears in Protocol F.1. The protocol works by generating a random sharing $[\![s]\!]_t$, and letting P_i fix its share to be r_i by broadcasting $r_i - s_i$. A malicious P_i can of course send a different value. To verify correctness we wish to take $[\![r_i|_i]\!]_{2t} - [\![r_i|_i]\!]_t$ and check that P_i's share is 0. To perform this check securely, the parties randomize their shares by adding a random sharing where P_i's share is 0. Note that if the final reconstruction fails, the parties call $\mathsf{ss.check}()$, which works only with a sharing that can be decomposed (see above). This indeed holds for the use of the protocol in our construction.

Protocol F.1. $\mathsf{ss.convert}([\![r_i|_i]\!]_{2t},i)$**: Conversion Protocol**

- **Input:** The parties hold a consistent secret sharing $[\![r_i|_i]\!]_{2t}$, where r_i is the share of P_i.
- **The protocol:**
  1. The parties generate a random sharing $[\![s]\!]_t$: each party P_j chooses a random s^j, runs $[\![s^j]\!]_t \leftarrow \mathsf{share}(s^j)$ with threshold t and sends each other party its share. Then, the parties locally compute $[\![s]\!]_t = \sum_{j=1}^{n} [\![s^j]\!]_t$.
     Let s_i be party P_i's share of s.
  2. The parties generate a random sharing $[\![v]\!]_{2t}$: each party P_j chooses a random v^j, runs $[\![v^j]\!]_{2t} \leftarrow \mathsf{share}(v^j)$ with threshold $2t$ and sends each other party its share. Then, the parties locally compute $[\![v]\!]_{2t} = \sum_{j=1}^{n} [\![v^j]\!]_{2t}$.
     Let v_i be party P_i's share of v.
  3. Party P_i broadcasts (via $\mathcal{F}_{\mathsf{bc}}$) v_i and $r_i - s_i$.
  4. The parties locally compute:
     - $[\![0|_i]\!]_{2t} = [\![v]\!]_{2t} - [\![v_i|_i]\!]_{2t}$
       where $[\![v_i|_i]\!]_{2t}$ is a public sharing defined by running $\mathsf{share}(0, \{v'_j\}_{j\in T})$ such that $T = \{j_1,...,j_{2t-1},i\}$ is a fixed set of size $2t$ and $v'_{j_1} = ... = v'_{j_{2t-1}} = 0$, $v'_i = v_i$.
     - $[\![r_i|_i]\!]_t = [\![s]\!]_t + [\![r_i - s_i|_i]\!]_t$
       where $[\![r_i - s_i|_i]\!]_t$ is a public sharing defined by running $\mathsf{share}(0, \{v'_j\}_{j\in T})$ such that $T = \{j_1,...,j_{t-1},i\}$ is a fixed set of size t and $v'_{j_1} = ... = v'_{j_{t-1}} = 0$, $v'_i = r_i - s_i$.
  5. The parties call $\mathcal{F}_{\mathsf{coin}}$ to receive α and locally compute $[\![u]\!]_{2t} = [\![r_i|_i]\!]_{2t} - [\![r_i|_i]\!]_t + \alpha \cdot [\![0|_i]\!]_{2t}$ (to perform this computation, the parties locally compute $[\![1|_i]\!]_t \cdot [\![r_i|_i]\!]_t$ and then carry out the addition).
  6. The parties run $\mathsf{reconstruct}([\![u]\!]_{2t})$ by broadcasting their shares to each other.
     - If $\mathsf{reconstruct}([\![u]\!]_{2t}) \neq \perp$ and P_i's share is 0, then the parties output $[\![r_i|_i]\!]_t$.
     - If $\mathsf{reconstruct}([\![u]\!]_{2t}) \neq \perp$ and P_i's share is *not* 0, then party P_i broadcasts an index k and the parties output the pair (i,k).
     - If $\mathsf{reconstruct}([\![u]\!]_{2t}) = \perp$, then the parties run $(i,k) \leftarrow \mathsf{ss.check}([\![u]\!]_{2t})$ and output (i,k).

Simulating the protocol. We next describe a simulator for the protocol that we will use in our security proof. There are two cases for which the simulation works differently as detailed below

- *Case 1: P_i is honest.* In this case, the simulator $\mathcal{S}$ knows the shares held by corrupted parties in $[\![r_i|_i]\!]_{2t}$. In the simulation, $\mathcal{S}$ simulates each of the share executions where the dealer is honest by choosing random shares for the corrupted parties and handing them to $\mathcal{A}$. When the dealer is corrupted, $\mathcal{S}$ receives the shares of the honest parties from $\mathcal{A}$. These are used to compute the corrupted parties' shares. To simulate the broadcast message from P_i, the simulator chooses random elements and hands them to $\mathcal{A}$. To simulate the reconstruct executions, $\mathcal{S}$ chooses random shares for the other honest parties such that all shares are consistent and the share of P_i is 0. If reconstruction fails, $\mathcal{S}$ runs the simulator of ss.check described in Section E and outputs a semi-corrupt pair. Note that the simulation is identically distributed as the real execution.

- *Case 2: P_i is corrupted.* In this case, $\mathcal{S}$ knows r_i and all the shares held by the parties in $[\![r_i|_i]\!]_{2t}$. Since the input is known, it can simply follow the protocol's instructions while playing the role of the honest parties. There is, however, one event where the simulation differs from a real execution: when $\mathcal{A}$ cheats and sends $r_i - s_i + \Delta$, making the output sharing be a sharing of $r_i + \Delta$ instead of r_i, and yet the honest parties accept the resulting sharing. In this case, the simulator outputs fail. Observe that this event happens when the share of P_i in $[\![u]\!]_{2t}$ is 0. Let P_i's share in $[\![0|_i]\!]_{2t}$ be z_i. Then, this event happens when $\Delta_i + \alpha \cdot z_i = 0$. By Lemma 2.2, for a ring R with a largest exceptional subset of size ω_R, this happens with probability $\frac{1}{\omega_R}$.

G Deferred Proofs from Section 6

G.1 Proof for Lemma 6.2

Lemma 6.2. Let $t_i = n/3 - 1 - i$ and $n_i = n - 2i$. That is, t_i is an upper bound on the number of corrupted parties left after i eliminations of pairs of parties and n_i is the number of parties left. We find i for which $t_i < n_i/3 - g(n_i)$. Since $n = n_i + 2i$, we have $t_i = n_i/3 + 2i/3 - 1 - i = n_i/3 - i/3 - 1$. Now, taking $i = 3g(n)$, we have $t' \leq n'/3 - g(n) - 1$. Since g is a monotonically increasing function, we have $g(n) > g(n')$ and so $t' \leq n'/3 - g(n')$. This means that t' has a gap of $g(n')$ from $n'/3$ as required. □

G.2 Proof for Lemma 6.3

Lemma 6.3. For the first part, let C_i be an indicator variable that is equal to 1 if the i-th ball in S is chosen into $\mathbf{X}$. Let $C = \sum_{i \in [n]} C_i$ and therefore $\mathbb{E}[C] = k(n)$. It follows that:

$$\Pr[|C - k| \geq 0.5k] \;=\; \Pr[|C - k| \geq 0.5\mathbb{E}[C]] \overset{(1)}{\leq} 2e^{\frac{-k}{12}} \overset{(2)}{\leq} 2^{-\sigma}$$

where inequality (1) follows from Chernoff bound, and inequality (2) holds as long as $k > \ln 2 \cdot 12 \cdot (\sigma + 1)$ as needed.

For the second part, let $S_{\mathbf{X}}$ be the random variable equal to the size of the random set $\mathbf{X}$. Also let B_i (for $i \in [t]$) denote the random variable that is 1 if the i-th red ball is in $\mathbf{X}$ and 0 otherwise. Let $1 \leq c \leq n$ be an integer. Once the size of the set $\mathbf{X}$ is fixed to c, we have $S_{\mathbf{X}} = c$, and the conditional expectation of B_i is $\mathbb{E}[B_i \,|\, S_{\mathbf{X}} = c] = |\mathbf{X}|/n = c/n$. Let $B = \sum_{i=1}^{t} B_i$ be the total number of red balls in $\mathbf{X}$.

Let $\mu=\mathbb{E}[B\,|\,S_{\mathbf{X}}=c]=(t\cdot c)/n$ be the expected number of red balls in $\mathbf{X}$. We would like to find a constraint on g for which $\Pr[B\geq c/3|S_{\mathbf{X}}=c]$ is negligible in n. Notice that $c/3=\mu\cdot\mu^{-1}\cdot c/3=(n/3t)\cdot\mu>\left(1+\frac{g}{t}\right)\mu$ where last transition holds since $t<n/3-g$ and so $1+\frac{g}{t}<\frac{n}{3t}$. Therefore, we have that

$$\Pr[B\geq c/3|S_{\mathbf{X}}=c]\overset{(1)}{\leq}\Pr\left[B\geq\left(1+\frac{g}{t}\right)\mu|S_{\mathbf{X}}=c\right]\overset{(2)}{\leq}e^{-\frac{(g/t)^2\cdot(tc)/n}{2+(g/t)}}\overset{(3)}{=}e^{-\frac{g^2\cdot c}{n(2t+g)}}\overset{(4)}{<}e^{-\frac{g^2\cdot c}{n^2}}$$

where the inequality (2) holds by applying the Chernoff bound and the last inequality holds by substituting g in the denominator by t and by substituting $3t$ with n. Therefore, given any σ if $g^2\cdot c\geq\ln(2)\sigma n^2$ then the probability is $<2^{-\sigma}$. So for any c, *given* that the size of the chosen set $\mathbf{X}$ is c, then the gap g must satisfy $g^2\cdot c\geq\ln(2)\sigma n^2$ for $\mathbf{X}$ to contain more than two-thirds of the red balls with probability $<2^{-\sigma}$.

To conclude the proof, we recall from part 1 of the lemma, that with all but negligible probability the size of the set $\mathbf{X}$ is in the range $[0.5k,1.5k]$. We notice that for each $c\in[0.5k,1.5k]$ we require that $g^2\cdot c\geq\ln(2)\sigma n^2$ for the subset to contain at least two-thirds fraction of red balls. Therefore, we can take the minimum of c in the range $[0.5k,1.5k]$ which is $0.5k$ and get that $g^2\cdot 0.5k\geq\ln(2)\sigma n^2$, or equivalently for $g=n\sqrt{2\ln(2)\sigma/k}$ and for any $c\in[0.5k,1.5k]$ it holds that $\Pr[|R\cap\mathbf{X}|\geq c/3\,|\,S_{\mathbf{X}}=c]\leq 2^{-\sigma}$. Consider choosing g such that $g^2\cdot k=2\ln(2)(\sigma+1)n^2$ (notice the added "+1" term), then:

$$\begin{aligned}
\Pr[|R\cap\mathbf{X}|\geq|\mathbf{X}|/3]&\overset{(1)}{=}\sum_{c\in[0,n]}\Pr[|R\cap\mathbf{X}|\geq c/3\wedge S_{\mathbf{X}}=c]\\
&\overset{(2)}{=}\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c/3\wedge S_{\mathbf{X}}=c]+\\
&\quad\sum_{c\in[0,0.5k)\cup(1.5k,n]}\Pr[|R\cap\mathbf{X}|\geq c/3\wedge S_{\mathbf{X}}=c]\\
&\overset{(3)}{\leq}\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c/3\wedge S_{\mathbf{X}}=c]+\\
&\quad\sum_{c\in[0,0.5k)\cup(1.5k,n]}\Pr[S_{\mathbf{X}}=c]\\
&\overset{(4)}{<}\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c/3\wedge S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
&\overset{(5)}{=}\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c/3\,|\,S_{\mathbf{X}}=c]\cdot\Pr[S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
&\overset{(6)}{<}\sum_{c\in[0.5k,1.5k]}2^{-(\sigma+1)}\cdot\Pr[S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
&\overset{(7)}{=}2^{-(\sigma+1)}\sum_{c\in[0.5k,1.5k]}\Pr[S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
&\overset{(8)}{\leq}2^{-(\sigma+1)}+2^{-(\sigma+1)}\\
&=2^{-\sigma}
\end{aligned}$$

Where (1) follows from law of total probability, (2) simply splits the sum range into two, (3) follows since for any two events A,B we have $\Pr[A\cap B]\leq\Pr[A]$. (4) from part (1) of our Lemma 6.3

where probability of a set $\mathbf{X}$ to be sampled out of the range $[0.5k,1.5k]$ is less than $2^{-(\sigma+1)}$ for the parameters we've chosen. Step (5) follows from Bayes rule. Step (6) follows from our choice of g as above. Step (7) is an algebraic simplification and step (8) follows from law of total probability. This concludes our proof. □

G.3 Proof for Theorem 6.10

Theorem 6.10. **Round Complexity.** The parties compute at most L segments, each of depth $\mathsf{depth}(C)/g(n)$. Hence, round complexity is $O(L\cdot(\mathsf{depth}(C)\cdot\mathsf{Round}(\pi_{\mathsf{mult}})/g(n))+L)$.

Communication Complexity. The total communication complexity is calculated by first evaluating $3g(n)$ segments in

$$O\big(|C|\cdot\mathsf{Comm}(\pi_{\mathsf{mult}})+g(n)\cdot n^2\cdot\log(|C|/(n\cdot g(n)))\big)$$

communication, then electing the size $k(n)$ committee and running additional $3g(k(n))$ segments in $O\Big(\frac{|C|\cdot\mathsf{Comm}(\pi_{\mathsf{mult}})\cdot g(k(n))}{g(n)}+g(k(n))\cdot k(n)^2\cdot\log(|C|/(n\cdot g(n)))\Big)$ communication, and so on. To provide slightly more detail on the term above:

- The $\frac{|C|\cdot\mathsf{Comm}(\pi_{\mathsf{mult}})\cdot g(k(n))}{g(n)}$ term comes from $g(k(n))$ executions of segments in the circuit, each of size $|C|/g(n)$ multiplication gates where each multiplication gate requires $\mathsf{Comm}(\pi_{\mathsf{mult}})$ communication to evaluate.
- The $g(k(n))\cdot k(n)^2\cdot\log(|C|/(n\cdot g(n)))$ comes from the $3g(k(n))$ eliminations of parties before another recursive committee election will take place. Each elimination requires the $k(n)$ parties to participate in a protocol requiring $k(n)^2\cdot\log(|C|/(n\cdot g(n)))$, which is exactly the protocol we described in Protocol 4.1.

Summing over ℓ recursive elections we get an upper bound of:

$$O\left(|C|\cdot\mathsf{Comm}(\pi_{\mathsf{mult}})\sum_{i=0}^{\ell-1}\left(\frac{g(k^i(n))}{g(n)}\right)+L\cdot n^2\log(|C|/(n\cdot g(n)))\right)$$

Computational Complexity. The overall computational complexity depends on the number of calls to π_{mult} and the number of segments evaluated times the cost of $\mathcal{F}_{\mathsf{vrfy}}$. Overall, the cost is $O(L\cdot(|C|/g(n))\cdot\mathsf{Comp}(\pi_{\mathsf{mult}})+L\cdot(|C|/g(n)))$ per party. □

G.4 Proof for Theorem 6.11

Theorem 6.11. Let n_i be the number of parties after eliminating i pairs of semi-corrupt parties. Thus, $n_i=n-2i$. Let t_i be the number of corrupted parties after eliminating i pairs of semi-corrupt parties. Thus,

$$\begin{aligned}
t_i &\le t-i\\
&=\left(\frac{1}{2}-h(n)\right)\cdot n-i-1\\
&=\left(\frac{1}{2}-h(n)\right)\cdot(n_i+2i)-i-1\\
&=\left(\frac{1}{2}-h(n)\right)\cdot n_i-2i\cdot h(n)-1
\end{aligned}$$

Therefore, to achieve a gap of $g(n)$ we need to find the minimum i such that:

$$\begin{aligned} t_i &\leq \left(\frac{1}{2}-h(n)\right)\cdot n_i - g(n) - 1 \\ \left(\frac{1}{2}-h(n)\right)\cdot n_i - 2i\cdot h(n) - 1 &\leq \left(\frac{1}{2}-h(n)\right)\cdot n_i - g(n) - 1 \\ -2i\cdot h(n) &\leq -g(n) \\ i &\geq \frac{g(n)}{2h(n)} \end{aligned}$$

This concludes the proof. □

G.5 Proof for Lemma 6.12

Lemma 6.12. The proof is similar to that of Lemma 6.3 with the main difference being in the expectation of red balls in the committee changing from $n/3-1-g(n)$ to $n(\frac{1}{2}-h(n))-g(n)$. Thus, the first item's proof in this lemma is identical to that of Lemma 6.3.

For the second part, let $S_{\mathbf{X}}$ be the random variable equal to the size of the random set $\mathbf{X}$. Also let B_i (for $i \in [t]$) denote the random variable that is 1 if the i-th red ball is in $\mathbf{X}$ and 0 otherwise. Let $1 \leq c \leq n$ be an integer. Once the size of the set $\mathbf{X}$ is fixed to c, we have $S_{\mathbf{X}} = c$, and the conditional expectation of B_i is $\mathbb{E}[B_i \mid S_{\mathbf{X}} = c] = |\mathbf{X}|/n = c/n$. Let $B = \sum_{i=1}^{t} B_i$ be the total number of red balls in $\mathbf{X}$. Let $\mu = \mathbb{E}[B \mid S_{\mathbf{X}} = c] = (t \cdot c)/n$ be the expected number of red balls in $\mathbf{X}$. We would like to find a constraint on g for which $\Pr[B \geq c\cdot(1/2-h(n)) | S_{\mathbf{X}} = c]$ is negligible in n. Notice that $c\cdot(1/2-h(n)) = \mu\cdot\mu^{-1}\cdot c\cdot(1/2-h(n)) = (n\cdot(1/2-h(n))/t)\cdot\mu > \left(1+\frac{g}{t}\right)\mu$ where last transition holds since $t < n\cdot(1/2-h(n))-g$ and so $1+\frac{g}{t} < \frac{n}{3t}$. Therefore, we have that

$$\begin{aligned} \Pr[B \geq c\cdot(1/2-h(n)) | S_{\mathbf{X}} = c] &\overset{(1)}{\leq} \Pr\left[B \geq \left(1+\frac{g}{t}\right)\mu | S_{\mathbf{X}} = c\right] \\ &\overset{(2)}{\leq} e^{-\frac{(g/t)^2\cdot(tc)/n}{2+(g/t)}} \\ &\overset{(3)}{=} e^{-\frac{g^2\cdot c}{n(2t+g)}} \\ &\overset{(4)}{<} e^{-\frac{g^2\cdot c}{n^2}} \end{aligned}$$

where the inequality (2) holds by applying the Chernoff bound and the last inequality holds since $2t+g < n$. Therefore, given any σ if $g^2\cdot c \geq \ln(2)\sigma n^2$ then the probability is $< 2^{-\sigma}$. So for any c, *given* that the size of the chosen set $\mathbf{X}$ is c, then the gap g must satisfy $g^2\cdot c \geq \ln(2)\sigma n^2$ for $\mathbf{X}$ to contain more than two-thirds of the red balls with probability $< 2^{-\sigma}$.

To conclude the proof, we recall from part 1 of the lemma, that with all but negligible probability the size of the set $\mathbf{X}$ is in the range $[0.5k, 1.5k]$. We notice that for each $c \in [0.5k, 1.5k]$ we require that $g^2\cdot c \geq \ln(2)\sigma n^2$ for the subset to contain at least two-thirds fraction of red balls. Therefore, we can take the minimum of c in the range $[0.5k, 1.5k]$ which is $0.5k$ and get that $g^2\cdot 0.5k \geq \ln(2)\sigma n^2$, or equivalently for $g = n\sqrt{2\ln(2)\sigma/k}$ and for any $c \in [0.5k, 1.5k]$ it holds that $\Pr[|R \cap \mathbf{X}| \geq c\cdot(1/2-h(n)) \mid S_{\mathbf{X}} = c] \leq 2^{-\sigma}$. Consider choosing g such that $g^2\cdot k = 2\ln(2)(\sigma+1)n^2$

(notice the added "+1" term), then:

$$
\begin{aligned}
&\Pr[|R\cap\mathbf{X}|\geq|\mathbf{X}|\cdot(1/2-h(n))]\\
\overset{(1)}{=}&\sum_{c\in[0,n]}\Pr[|R\cap\mathbf{X}|\geq c\cdot(1/2-h(n))\wedge S_{\mathbf{X}}=c]\\
\overset{(2)}{=}&\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c\cdot(1/2-h(n))\wedge S_{\mathbf{X}}=c]+\\
&\sum_{c\in[0,0.5k)\cup(1.5k,n]}\Pr[|R\cap\mathbf{X}|\geq c\cdot(1/2-h(n))\wedge S_{\mathbf{X}}=c]\\
\overset{(3)}{\leq}&\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c\cdot(1/2-h(n))\wedge S_{\mathbf{X}}=c]+\\
&\sum_{c\in[0,0.5k)\cup(1.5k,n]}\Pr[S_{\mathbf{X}}=c]\\
\overset{(4)}{<}&\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c\cdot(1/2-h(n))\wedge S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
\overset{(5)}{=}&\sum_{c\in[0.5k,1.5k]}\Pr[|R\cap\mathbf{X}|\geq c\cdot(1/2-h(n))\,|\,S_{\mathbf{X}}=c]\cdot\Pr[S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
\overset{(6)}{<}&\sum_{c\in[0.5k,1.5k]}2^{-(\sigma+1)}\cdot\Pr[S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
\overset{(7)}{=}&2^{-(\sigma+1)}\sum_{c\in[0.5k,1.5k]}\Pr[S_{\mathbf{X}}=c]+2^{-(\sigma+1)}\\
\overset{(8)}{\leq}&2^{-(\sigma+1)}+2^{-(\sigma+1)}\\
=&2^{-\sigma}
\end{aligned}
$$

Where (1) follows from law of total probability, (2) simply splits the sum range into two, (3) follows since for any two events A,B we have $\Pr[A\cap B]\leq\Pr[A]$. (4) from part (1) of our Lemma 6.3 where probability of a set $\mathbf{X}$ to be sampled out of the range $[0.5k,1.5k]$ is less than $2^{-(\sigma+1)}$ for the parameters we've chosen. Step (5) follows from Bayes rule. Step (6) follows from our choice of g as above. Step (7) is an algebraic simplification and step (8) follows from law of total probability. This concludes our proof. □